

EXP.NO	Title	Page No
3 a)	Shell script a) Arithmetic Operation -using expr command b) Check leap year using if-else	8 9
3 b)	a) Reverse the number using while loop b) Fibonacci series using for loop	10 11
4	Text processing using Awk script a) Employee average pay b) Results of an examination	13 15
5	System calls –fork(), exec(), getpid(),opendir(), readdir()	17
6a	FCFS	19
6b	SJF	23
6c	Priority	26
6d	Round Robin	29
7.	Inter-process Communication using Shared Memory	34
8	Producer Consumer using Semaphores	42
9	Bankers Deadlock Avoidance algorithms	54
10 a	Best Fit	57
10 b	First Fit	60

11a	FIFO	62
11b	LRU	67
11c	Optimal	70
12	File Organization Technique- single and Two level directory	73

♦ 1. Basic Vim Editor Operations

To delete all (execute after insertion and pressing esc button on keyboard)

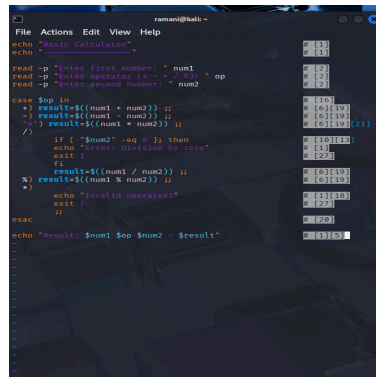
Action	Vim Command	Notes
Select All	<code>ggVG</code>	<code>gg</code> to top, <code>V</code> starts visual mode (linewise), <code>G</code> to bottom
Copy All	<code>:%y</code>	Copies all lines into the default register (not system clipboard) (not out of vim editor)
Delete All	<code>:%d</code>	Deletes all lines in the file
Go to Top	<code>gg</code>	Moves cursor to the first line
Go to Bottom	<code>G</code>	Moves cursor to the last line

Explanation:

- `gg`: Go to first line
- `V`: Visual mode (line selection)
- `G`: Go to last line from the current cursor position

2. How to Select Vertically (Visual Block Mode)

Example image for selection ;



Go to Normal mode: Press **Esc** if you're not already in normal mode.

Enter Visual Block mode: Press **Ctrl + v**

Move the cursor using arrows or **kjhl** for movement.

Press **d** to delete.

♦ 3. Copying from Vim to System Clipboard

✓ Checking Clipboard Support

```
vim --version | grep clipboard
```

- **+clipboard** → Clipboard support is **enabled**
- **-clipboard** → Clipboard support is **not enabled**

🔧 Enable Clipboard Support on Ubuntu/Debian

```
sudo apt install vim-gtk3
```

This version of Vim includes clipboard support (**+clipboard**).

♦ Copying to System Clipboard in Vim

To copy from the first line after pressing esc button on keyboard just go to the first line using mouse and enter with the command starting “

Action	Vim Command	Explanation
Copy current line	<code>"+yy</code>	<code>+</code> → system clipboard register, <code>yy</code> → yank current line
Copy 23 lines from current	<code>"+23yy</code>	Replace <code>23</code> with a large number to ensure copying all lines
Copy entire file	<code>gg"+yG</code>	Go to top, yank to clipboard till end (<code>yG</code>)
Copy selected (Visual Mode)	<code>v</code> → select → <code>"+y</code>	Select using <code>v</code> , then yank to system clipboard

♦ Extra Useful Vim Tips

Action	Command	Notes
Undo last change	<code>u</code>	
Redo last undone change	<code>Ctrl + r</code>	
Save file	<code>:w</code>	
Quit	<code>:q</code>	
Save and quit	<code>:wq</code> or <code>ZZ</code>	<code>ZZ</code> is a shortcut (shift + zz)
Quit without saving	<code>:q!</code>	
Search for a word	<code>/word</code>	Press <code>n</code> to go to the next match
Replace all	<code>:%s/old/new/ g</code>	Replaces all instances in the file
Enable line numbers	<code>:set number</code>	
Open multiple files	<code>vim file1 file2</code>	Use <code>:n</code> and <code>:prev</code> to navigate

◆ Bash & Shell Script Essentials (Short-Term Notes)

#	Concept	Description / Syntax
1	echo Statement	<code>echo "text"</code> — Prints text to the screen.
2	read Command	<code>read -p "Prompt" variable</code> — Takes user input. No <code>\$</code> when assigning.
3	Comments	<code># comment</code> — Single line comment.
4	Variables	<code>var=something</code> — ! No spaces around <code>=</code> .
5	Using Variables	<code>echo \$var</code> — Use <code>\$</code> to access value.
6	Arithmetic	<code>=\$((a + b))</code> — Arithmetic expressions.
7	Arithmetic Checks	<code>if [\$a -eq 5]</code> — Use <code>-eq</code> , <code>-ne</code> , etc. for numbers. No quotes.
8	String Comparison	<code>=</code> or <code>!=</code> inside <code>[]</code> . Use quotes: <code>"\$var" = "abc"</code> .
9	Null Check in if	<code>if ["\$val" = "abc"]</code> — Quotes prevent word splitting errors.
10	Conditional (if)	<code>if [condition]; then ... fi</code> — Always space inside <code>[]</code> .
11	Command Separation in if	<code>if [\$x -gt 1]; then</code> OR use newline after <code>]</code> .
12	else / elif	Standard flow: <code>elif</code> , <code>else</code> , <code>fi</code> .

- 13 **Loop (for)** `for var in list; do ...`
 `done`

- 14 **Loop (while)** `while [ condition ]; do`
 `... done`

- 15 **Loop with Numbers** `for i in {1..10} or seq`
 `1 10`

- 16 **case Statement** `case $var in pattern) ... ;; esac` — Like
 switch-case.

- 17 **Start of case** Use `in` to define: `case $input in`

- 18 **case Pattern Syntax** `1) or 'yes' )` — Pattern matching.

- 19 **End of case Pattern** End each pattern block with `;;` (one space before it).

- 20 **End of case Block** Close block with `esac`.

- 21 **Wildcard Matching** Use `'*'` to match any pattern in case. Must be in single
 quotes.

- 22 **Script Arguments** `$1, $2, ...` — Positional arguments from command line.

- 23 **Print All Arguments** `echo $@` — Prints all passed arguments.

- 24 **Run Script** `bash script.sh` or `./script.sh` (after `chmod`
 `+x`).

- 25 **Make Executable** `chmod +x
filename`

- 26 **Shell Shebang** First line: `#!/bin/bash` — Defines interpreter.

- 27 **Exit Command** `exit 0` — Ends script with status.

- 28 **Debugging** `bash -x script.sh` — Shows step-by-step execution.

- 29 **Check File Exists** `if [ -f filename ]` — Checks if file exists.

- 30 **Check Directory Exists** `if [ -d dirname ]` — Checks if folder exists.

- 31 **Multi-line Strings** `cat <<EOF ... EOF` — Used for heredocs.

- 32 **awk Usage** `awk -f script.awk
file.dat`

- 33 **Input Redirection** `awk -f script.awk <
file.dat`

- 34 **BEGIN in AWK** `BEGIN {}` — Runs before processing lines.

- 35 **AWK Columns** `$1, $2` — Represent columns in current line.

- 36 **.dat File** Plain text used for structured data.

- 37 **.awk File** AWK scripts using pattern/action format.

- 38 **AWK Blocks** `BEGIN {}, {}, END {}` — All are optional.

3a)

a1) shell script

To write a Shellscrip to to display basic calculator.

Code

description number

```

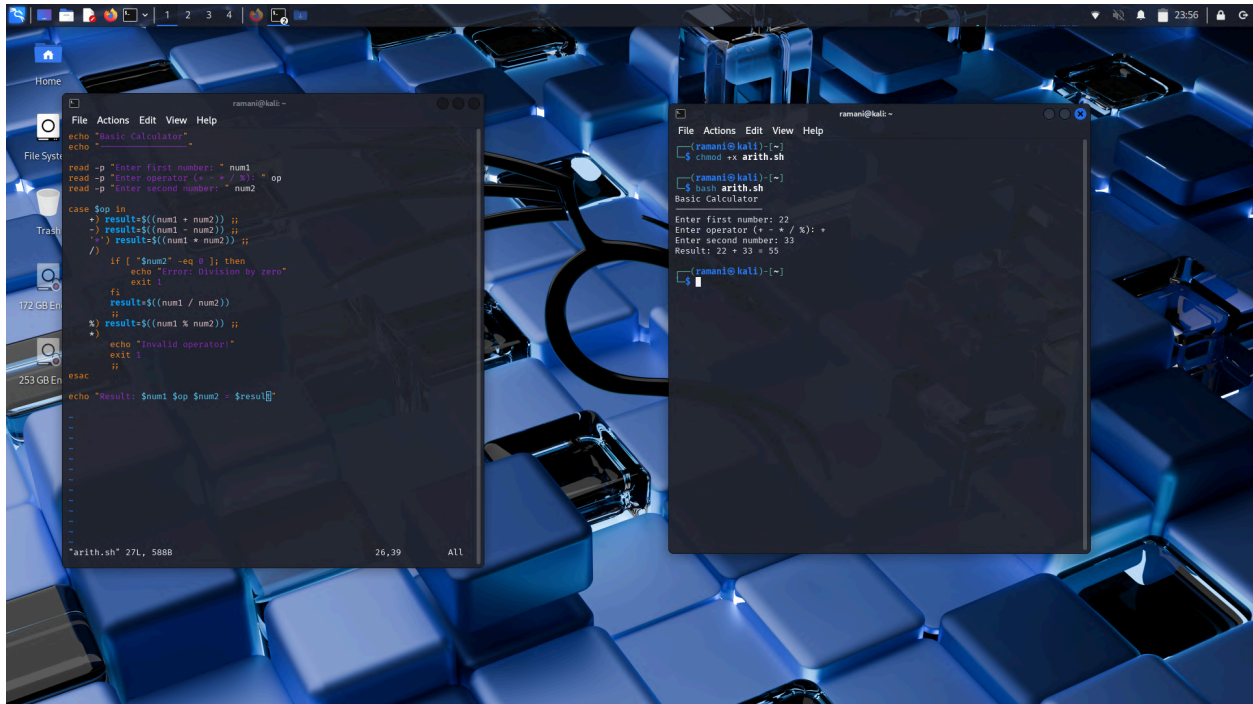
echo "Basic Calculator"                                # [1]
echo "-----"                                         # [1]

read -p "Enter first number: " num1                    # [2]
read -p "Enter operator (+ - * / %): " op              # [2]
read -p "Enter second number: " num2                  # [2]

case $op in
+ ) result=$((num1 + num2)) ;;                          # [16]
- ) result=$((num1 - num2)) ;;                          # [6][19]
'%' ) result=$((num1 * num2)) ;;                       # [6][19][21]
/)
    if [ "$num2" -eq 0 ]; then                          # [10][13]
        echo "Error: Division by zero"                 # [1]
        exit 1                                          # [27]
    fi
    result=$((num1 / num2)) ;;                          # [6][19]
%) result=$((num1 % num2)) ;;                          # [6][19]
*)
    echo "Invalid operator!"                            # [1][18]
    exit 1                                              # [27]
;;
esac                                                    # [20]

echo "Result: $num1 $op $num2 = $result"              # [1][5]

```

b) qn: To write a Shellsript to test given year is leap or not using conditional statement

note change the code to bash

\$ bash [filename.sh](#)

Code:

description number

```
# Ask user for the year
read -p "Enter a year: " year
```

[2]

```
# Check if the year is a leap year
if (( year % 400 == 0 )); then
    echo "yes"
elif (( year % 100 == 0 )); then
    echo "no"
elif (( year % 4 == 0 )); then
    echo "yes"
else
    echo "no"
fi
```

[10][6]

[1]

[10][6]

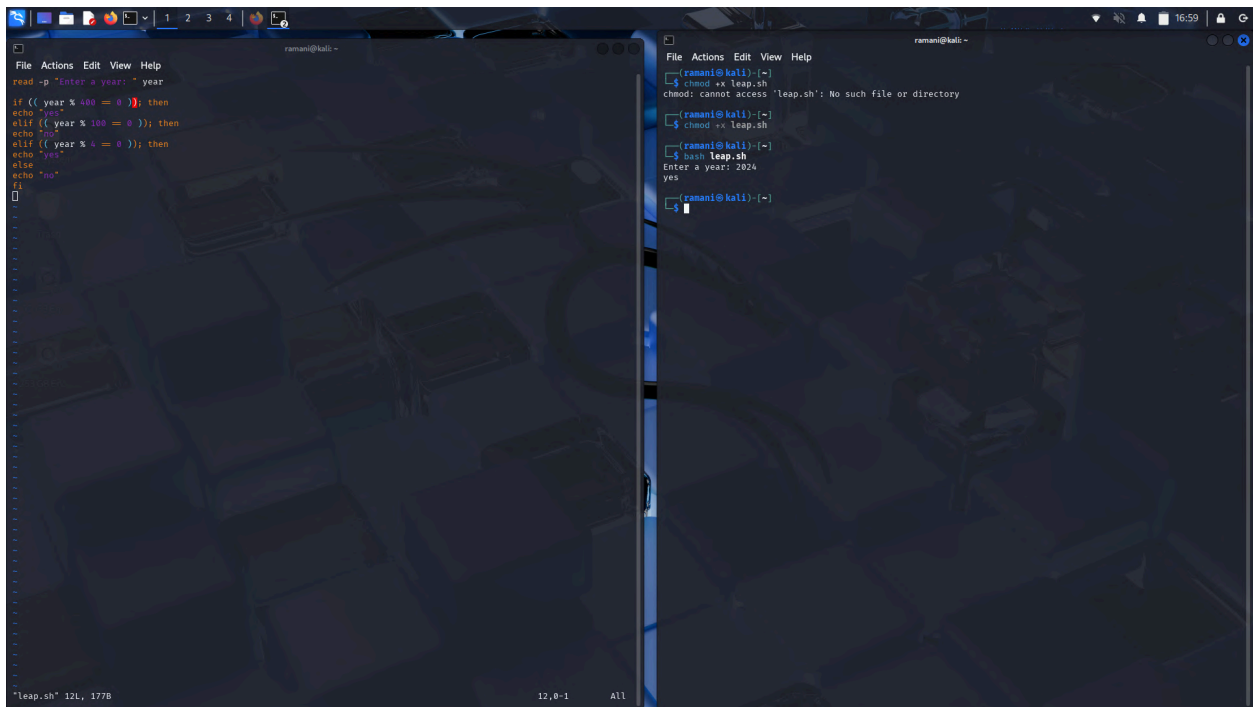
[1]

[10][6]

[1]

[1]

[15]



3b)

a2)

qn: reverse a number using shellsript

Note:

Use bash

\$bash "your execution"

code;

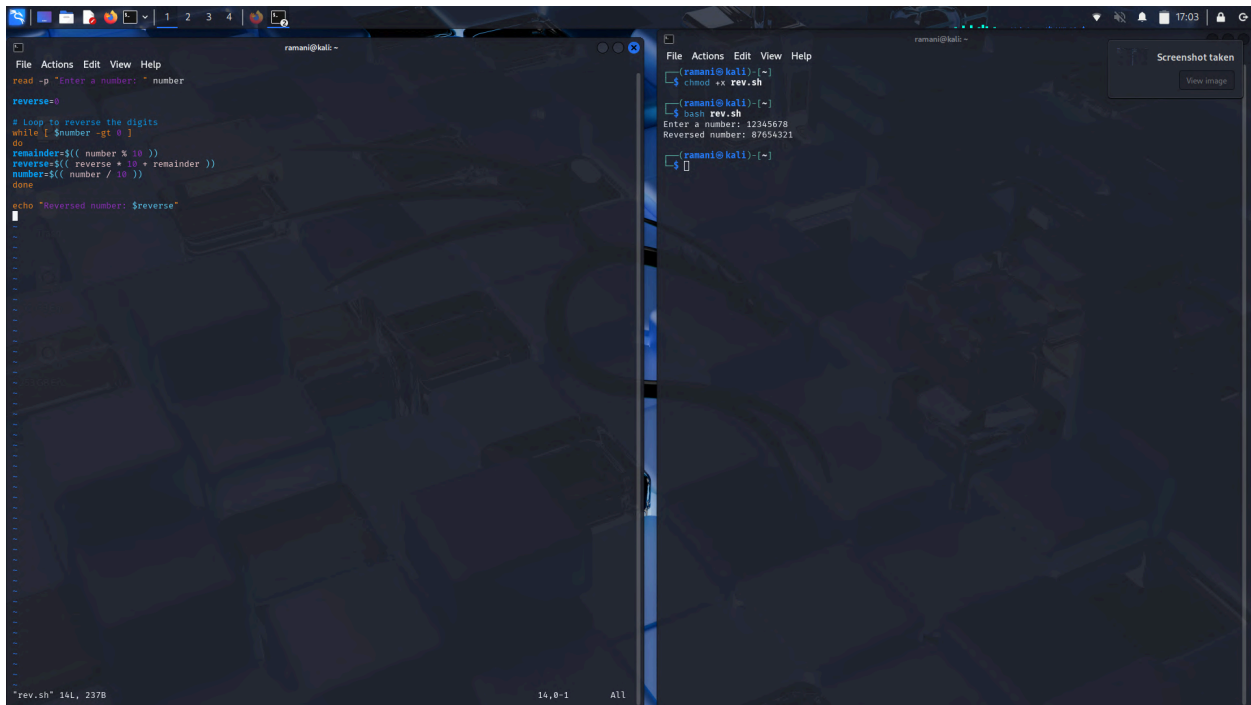
```
# Read a number from the user
read -p "Enter a number: " number # [2]

reverse=0 # [4]

# Loop to reverse the digits
while [ $number -gt 0 ] # [12][10]
do # [11]
    remainder=$(( number % 10 )) # [6]
    reverse=$(( reverse * 10 + remainder )) # [6]
    number=$(( number / 10 )) # [6]
```

```
done # [11]
```

```
echo "Reversed number: $reverse" # [1][5]
```



b2)

qn : generate fibonacci series using for loop in shell script

Code:

```
read -p "Enter the number of terms: " n # [2]
```

```
a=0 # [4]
```

```
b=1 # [4]
```

```
echo "Fibonacci series up to $n terms:" # [1][5]
```

```
for (( i=0; i<n; i++ )) # [11][6]
```

```
do # [1][5]
```

```
    echo -n "$a "
```

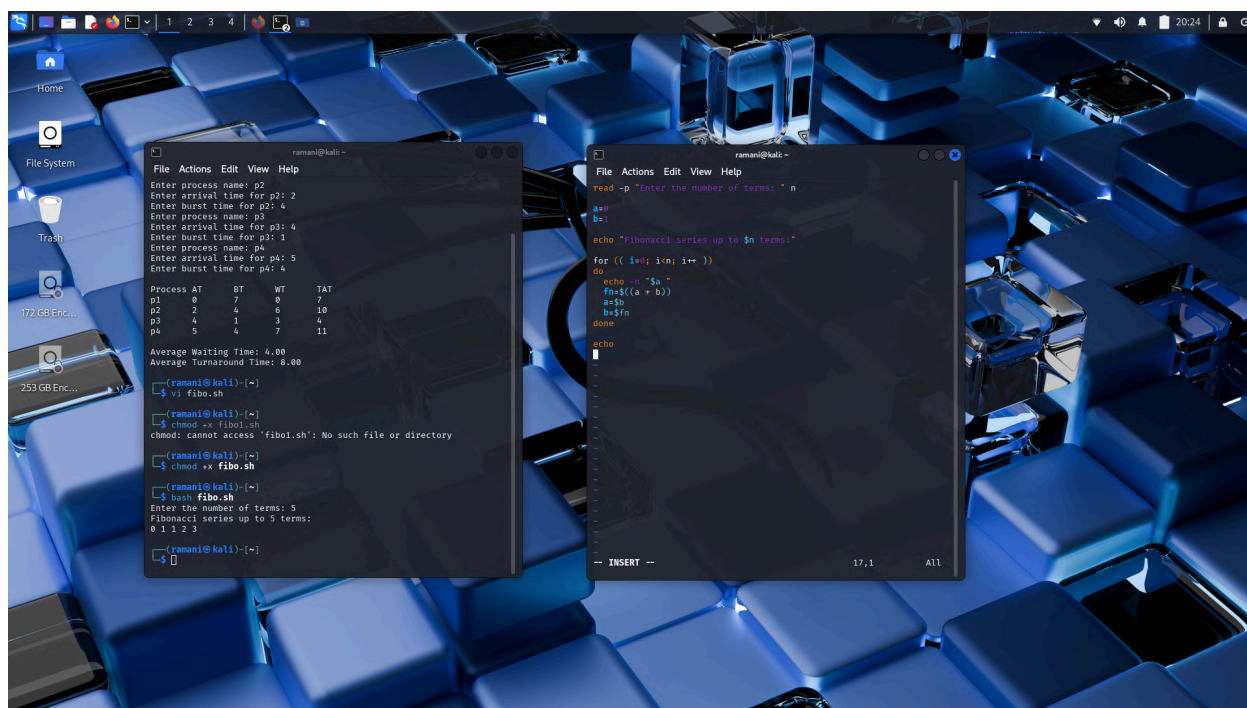
```

fn=$((a + b))           # [6]
a=$b                    # [4]
b=$fn                   # [4]
done                     # [11]

echo                     # [1]

```

note
use bash



4) text processing using Awk script

4)

a)

EMPLOYEE AVERAGE PAY

Aim:

To find out the average pay of all employees whose salary is more than 6000 and no. of days worked is more than 4.

Algorithm:

1. Create a flat file emp.dat for employees with their name, salary per day and number of days worked and save it.
2. Create an awk script emp.awk
3. For each employee record do
 - a. If Salary is greater than 6000 and number of days worked is more than 4, then print name and salary earned
 - b. Compute total pay of employee
4. Print the total number of employees satisfying the criteria and their average pay.

Answer:

step1)

Create a file with the name emp.data to store the data

step2)

Create a file with the name emp.awk to process and analyze text file or data file

step3)

Run the awk script using the line `awk -f emp.awk emp.data`

Code :

```
$vi emp.data
```

```
John 1000 7
Alice 900 3
Bob 950 6
Rita 700 5
David 1200 6
Eve 1100 4
```

\$vi emp.awk

```

BEGIN {
    totalPay = 0;
    count = 0;
    print "Employees with Salary > 6000 and Days Worked > 4:";
[1] print "-----";
}

{
    salary = $2 * $3;
    if (salary > 6000 && $3 > 4) {
    print $1, "=>", salary;
    totalPay += salary;
    count++;
    }
}

END {
    if (count > 0) {
    avgPay = totalPay / count;
    print "-----";
    print "Total employees:", count;
    print "Average Pay:", avgPay;
    } else {
    print "No employees found with criteria.";
    }
}

```

\$awk -f emp.awk emp.data

```

Name SalaryPerDay DaysWorked
John 1000 7
Alice 900 3
Bob 950 6
Kita 700 5
David 1200 6
Eve 1100 4

BEGIN {
    totalPay = 0;
    count = 0;
    print "Employees with Salary > 6000 and Days Worked > 4:";
    print "-----";
    {
        salary = $2 * $3;
        if (salary > 6000 && $3 > 4) {
            count++;
            totalPay += salary;
        }
    }
    END {
        if (count > 0) {
            avgPay = totalPay / count;
            print "Total employees: ", count;
            print "Average Pay: ", avgPay;
        } else {
            print "No employees found with criteria.";
        }
    }
}

```

```

John => 7000
David => 7200

Total employees: 2
Average Pay: 7100

```

4b)

Aim:

To print the pass/fail status of a student in a class.

Algorithm:

1. Read the data from file
2. Get a data from each column
3. Compare the all subject marks column
 - a. If marks less than 45 then print Fail
 - b. else print Pass

Answer;

- Step 1: Create the Data File (**emp.data**)
- Step 2: Create the AWK Script (emp.awk)
- Step 3: Running the AWK Script

Program Code:

```
$vi emp.data
```

```
BEN Manager 60000
TOM Developer 55000
RAM Tester 48000
JIM HR 52000
ALICE Developer 57000
```

```
$vi emp.awk
```

```
BEGIN {                                                    # [19][23]
    print "NAME    SUB-1 SUB-2 SUB-3 SUB-4 SUB-5 SUB-6 STATUS" # [1]
    print "-----" # [1]
}

NF == 7 {                                                    # [10]
    name = $1                                                # [20]
    status = "PASS"
    for (i = 2; i <= 7; i++) {                                # [11]
        if ($i < 45)                                          # [10][20]
            status = "FAIL"
    }
    printf "%-6s %5d %5d %5d %5d %5d %5d %7s\n", name, $2, $3, $4, $5, $6,
$7, status # [1][20]
}

NF != 7 && NF > 0 {                                           # [10][30]
    print "[Warning] Skipped invalid or incomplete line: " $0 # [1][20]
}
```

```
$ gawk -f emp.awk emp.data
```



```

BEGIN {
    print "NAME SUB-1 SUB-2 SUB-3 SUB-4 SUB-5 SUB-6 STATUS"
}

NF == 7 {
    name = $1
    status = "PASS"
    for (i = 2; i <= 7; i++) {
        if ($i < 45)
            status = "FAIL"
    }
    printf "%-6s %5d %5d %5d %5d %5d %5d\n", name, $2, $3, $4, $5, $6, $7,
    status
}

NF != 7 || NF > 0 {
    print "[Warning] Skipped invalid or incomplete line: " $0
}

```

NAME	SUB-1	SUB-2	SUB-3	SUB-4	SUB-5	SUB-6	STATUS
BEN	40	55	66	77	55	77	FAIL
TOM	60	67	84	92	98	68	PASS
RAM	90	95	84	87	56	78	PASS
JIM	60	70	65	78	90	87	PASS
	0	0	0	0	0	0	FAIL

5) system call programming

experiment with system calls using fork(), execlp() and pid() functions.

Code:

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

```

```

int main() {
    int pid;
    pid = fork(); // Create new process

    printf("THIS LINE EXECUTED TWICE\n");

    if (pid == -1) {
        printf("CHILD PROCESS NOT CREATED\n");
        exit(0);
    }

    if (pid == 0) {
        // Child process block
        printf("Child Process ID: %d\n", getpid());
    }
}

```



6.

Display the total waiting time, average waiting time, turnaround time

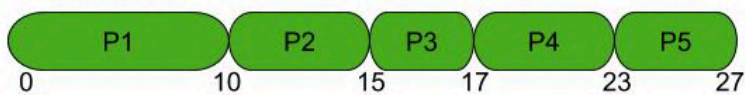
To implement First-come First-serve (FCFS) scheduling technique

Explanation;

Example;

PROCESS	ARRIVAL TIME	BURST TIME
P1	0	10
P2	3	5
P3	5	2
P4	6	6
P5	8	4

GANTT CHART



Code explanation;

Struct process;

```
struct Process {
    char name[10];           // To store the name (ID) of the process (e.g., "P1")
    int burstTime;           // Time required for the process to complete
    int waitingTime;         // Time the process has to wait before it starts execution
    int turnaroundTime;      // Total time from submission to completion (waiting + burst)
};
```



VARIABLES USED IN MAIN

- `int n`: Number of processes
- `struct Process p[10]`: Array to store up to 10 processes and their data

- `int totalWaitingTime, totalTurnaroundTime`: For computing total and average times
- `float avgWT, avgTAT`: To store average Waiting and Turnaround Time

2. FCFS Scheduling Calculation

a. For the first process:

`p[0].waitingTime = 0;`

`p[0].turnaroundTime = p[0].burstTime;`

1. Waiting time is zero (no one before it).
2. Turnaround time = time to finish its burst.

For the result;

`p[i].waitingTime = p[i - 1].waitingTime + p[i - 1].burstTime;`

`p[i].turnaroundTime = p[i].waitingTime + p[i].burstTime;`

1. Each process waits for all previous ones to finish.
2. Turnaround = its own burst time + how long it waited.

Code;

```
#include <stdio.h>
```

```
struct Process {
    char name[10];
    int burstTime;
    int waitingTime;
    int turnaroundTime;
};
```

```
int main() {
    int n, i;
    struct Process p[10];
    int totalWaitingTime = 0, totalTurnaroundTime = 0;

    printf("Enter the number of processes: ");
    scanf("%d", &n);

    // Input process name and burst time
    for (i = 0; i < n; i++) {
        printf("Enter process name: ");
        scanf("%s", p[i].name);
        printf("Enter burst time for %s: ", p[i].name);
        scanf("%d", &p[i].burstTime);
    }

    // FCFS scheduling
    p[0].waitingTime = 0;
    p[0].turnaroundTime = p[0].burstTime;

    for (i = 1; i < n; i++) {
        p[i].waitingTime = p[i - 1].waitingTime + p[i - 1].burstTime;
        p[i].turnaroundTime = p[i].waitingTime + p[i].burstTime;
    }

    // Display process info
    printf("\nProcess\tBurst Time\tWaiting Time\tTurnaround Time\n");
    for (i = 0; i < n; i++) {
        printf("%s\t%d\t\t%d\t\t%d\n", p[i].name, p[i].burstTime, p[i].waitingTime,
p[i].turnaroundTime);
        totalWaitingTime += p[i].waitingTime;
        totalTurnaroundTime += p[i].turnaroundTime;
    }
}
```

```

// Averages
float avgWT = (float)totalWaitingTime / n;
float avgTAT = (float)totalTurnaroundTime / n;

printf("\nTotal Waiting Time: %d", totalWaitingTime);
printf("\nAverage Waiting Time: %.2f", avgWT);
printf("\nTotal Turnaround Time: %d", totalTurnaroundTime);
printf("\nAverage Turnaround Time: %.2f\n", avgTAT);

return 0;
}

```

The screenshot shows a terminal window with the following output:

```

ramani@kali:~$ ./fcts2.c
Enter the number of processes: 3
Enter process name: p1
Enter burst time for p1: 3
Enter process name: p2
Enter burst time for p2: 4
Enter process name: p3
Enter burst time for p3: 6

```

Process	Burst Time	Waiting Time	Turnaround Time
p1	3	0	3
p2	4	3	7
p3	6	7	13

```

Total Waiting Time: 10
Average Waiting Time: 3.33
Total Turnaround Time: 23
Average Turnaround Time: 7.67

```

6)

b)

Give arrival time = 0 for each process as from the example of lab manual

Aim:

To implement the Shortest Job First (SJF) scheduling technique

Non-preemptive Shortest Job First

- Example:

Process:	p1	p2	p3	p4
Arrival time:	0	2	4	5
Burst time:	7	4	1	4
- Schedule:

P1	P3	P2	P4
----	----	----	----

7
8
12
16

Waiting time: 0 6 3 7

Average waiting time: 4.

```
#include <stdio.h>
```

```
#include <string.h>
```

```
struct Process {
    char name[10];
    int arrivalTime;
    int burstTime;
```

```

    int waitingTime;
    int turnaroundTime;
    int completed;
};

int main() {
    int n, time = 0, completed = 0;
    struct Process p[10];
    float totalWT = 0, totalTAT = 0;

    printf("Enter the number of processes: ");
    scanf("%d", &n);

    // Step 3: Input process info
    for (int i = 0; i < n; i++) {
        printf("Enter process name: ");
        scanf("%s", p[i].name);
        printf("Enter arrival time for %s: ", p[i].name);
        scanf("%d", &p[i].arrivalTime);
        printf("Enter burst time for %s: ", p[i].name);
        scanf("%d", &p[i].burstTime);
        p[i].completed = 0; // Step 4: Initialization
    }

    // Step 6: SJF Logic
    while (completed < n) {
        int idx = -1;
        int minBurst = 9999;

        for (int i = 0; i < n; i++) {
            if (p[i].arrivalTime <= time && !p[i].completed) {
                if (p[i].burstTime < minBurst) {
                    minBurst = p[i].burstTime;
                    idx = i;
                }
            }
        }

        if (idx != -1) {
            p[idx].waitingTime = time - p[idx].arrivalTime;
            p[idx].turnaroundTime = p[idx].waitingTime + p[idx].burstTime;
            time += p[idx].burstTime;
            p[idx].completed = 1;
            completed++;
        }
    }
}

```



```

totalWT += p[idx].waitingTime;
totalTAT += p[idx].turnaroundTime;
} else {
time++; // CPU idle if no process is ready
}
}

// Step 8: Display results
printf("\nProcess\tAT\tBT\tWT\tTAT\n");
for (int i = 0; i < n; i++) {
printf("%s\t%d\t%d\t%d\t%d\n", p[i].name, p[i].arrivalTime, p[i].burstTime,
p[i].waitingTime, p[i].turnaroundTime);
}

printf("\nAverage Waiting Time: %.2f", totalWT / n);
printf("\nAverage Turnaround Time: %.2f\n", totalTAT / n);

return 0;
}

```

The screenshot shows a terminal window with the following output:

```

ramani@kali:~$ ./s3f1.c
Enter the number of processes: 4
Enter process name: p1
Enter arrival time for p1: 0
Enter burst time for p1: 8
Enter process name: p2
Enter arrival time for p2: 4
Enter burst time for p2: 4

ramani@kali:~$ ./s3f1.c
Enter the number of processes: 4
Enter process name: p1
Enter arrival time for p1: 0
Enter burst time for p1: 8
Enter process name: p2
Enter arrival time for p2: 1
Enter burst time for p2: 1
Enter process name: p3
Enter arrival time for p3: 2
Enter burst time for p3: 2
Enter process name: p4
Enter arrival time for p4: 3
Enter burst time for p4: 1

Process AT   BT   WT   TAT
p1  0   8   0   8
p2  1   4  10  14
p3  2   2   7   9
p4  3   1   5   6

Average Waiting Time: 5.50
Average Turnaround Time: 9.25

ramani@kali:~$ ./s3f1.c
Enter the number of processes: 3
Enter process name: p1
Enter arrival time for p1: 0
Enter burst time for p1: 6
Enter process name: p2
Enter arrival time for p2: 1
Enter burst time for p2: 8
Enter process name: p3
Enter arrival time for p3: 2
Enter burst time for p3: 7

Process AT   BT   WT   TAT
p1  0   6   0   6
p2  1   8  12  20
p3  2   7   4  11

Average Waiting Time: 5.33
Average Turnaround Time: 12.33

```

6)

c)

PRIORITY SCHEDULING

Aim:

To implement priority scheduling technique

Code :

#include <stdio.h>

#include <string.h>

```

struct Process {
    char name[10];
    int burstTime;
    int priority;
    int waitingTime;
    int turnaroundTime;
};

```

```

int main() {
    int n, count = 0;
    struct Process p[10], filtered[10];
    int totalWT = 0, totalTAT = 0;

    printf("Enter the number of processes: ");
    scanf("%d", &n);

    // Step 2: Read process info
    for (int i = 0; i < n; i++) {
        printf("\nEnter process name: ");
        scanf("%s", p[i].name);
        printf("Enter burst time for %s: ", p[i].name);
        scanf("%d", &p[i].burstTime);
        printf("Enter priority for %s: ", p[i].name);
        scanf("%d", &p[i].priority);

        // Step 3: Filter priority == 4
        if (p[i].priority == 4) {
            filtered[count++] = p[i];
        }
    }

    // Sort by burst time (ascending)
    for (int i = 0; i < count - 1; i++) {

```

```

    for (int j = i + 1; j < count; j++) {
        if (filtered[i].burstTime > filtered[j].burstTime) {
            struct Process temp = filtered[i];
            filtered[i] = filtered[j];
            filtered[j] = temp;
        }
    }
}

// Step 4: Calculate waiting and turnaround time
int time = 0;
for (int i = 0; i < count; i++) {
    filtered[i].waitingTime = time;
    filtered[i].turnaroundTime = filtered[i].waitingTime + filtered[i].burstTime;
    time += filtered[i].burstTime;
    totalWT += filtered[i].waitingTime;
    totalTAT += filtered[i].turnaroundTime;
}

// Step 5: Display process info
printf("\nProcess\tBurst Time\n");
for (int i = 0; i < count; i++) {
    printf("%s\t%d\n", filtered[i].name, filtered[i].burstTime);
}

// Step 6: Final results
printf("\nTotal Waiting Time: %d", totalWT);
printf("\nAverage Waiting Time: %.2f", (float)totalWT / count);
printf("\nTotal Turnaround Time: %d", totalTAT);
printf("\nAverage Turnaround Time: %.2f\n", (float)totalTAT / count);

return 0;
}

```

```

File Actions Edit View Help
#include <iostream>
#include <string.h>

struct Process {
    char name[10];
    int burstTime;
    int priority;
    int waitingTime;
    int turnaroundTime;
};

int main() {
    int n, count = 0;
    struct Process p[10], filtered[10];
    int totalWT = 0, totalTAT = 0;

    printf("Enter the number of processes: ");
    scanf("%d", &n);

    // Step 1: Read process info
    for (int i = 0; i < n; i++) {
        printf("Enter process name: ");
        scanf("%s", p[i].name);
        printf("Enter burst time for %s: ", p[i].name);
        scanf("%d", &p[i].burstTime);
        printf("Enter priority for %s: ", p[i].name);
        scanf("%d", &p[i].priority);

        // Step 2: Filter priority = 4
        if (p[i].priority == 4) {
            filtered[count++] = p[i];
        }

        // Sort by burst time (ascending)
        for (int i = 0; i < count; i++) {
            for (int j = i + 1; j < count; j++) {
                if (filtered[i].burstTime > filtered[j].burstTime) {
                    struct Process temp = filtered[i];
                    filtered[i] = filtered[j];
                    filtered[j] = temp;
                }
            }
        }

        // Step 3: Calculate waiting and turnaround time
        int time = 0;
        for (int i = 0; i < count; i++) {
            filtered[i].waitingTime = time;
            filtered[i].turnaroundTime = filtered[i].waitingTime + filtered[i].burstTime;
            time += filtered[i].burstTime;
            totalWT += filtered[i].waitingTime;
            totalTAT += filtered[i].turnaroundTime;
        }

        // Step 4: Display process info
        printf("\nProcess Details:\n");
        for (int i = 0; i < count; i++) {
            printf("%s\t%d\t", filtered[i].name, filtered[i].burstTime);
        }

        // Step 5: Final results
        printf("\nTotal Waiting Time: %d, totalWT: ", (float)totalWT / count);
        printf("Average Waiting Time: %.2f, (float)totalWT / count);
        printf("\nTotal Turnaround Time: %d, totalTAT: ", (float)totalTAT / count);
        printf("Average Turnaround Time: %.2f\n", (float)totalTAT / count);

        return 0;
    }
}

```

```

File Actions Edit View Help
ramani@kali:~$ vi sjf
ramani@kali:~$ vi sjf.c
ramani@kali:~$ gcc sjf.c -o sjf.c
ramani@kali:~$ vi prio.c
ramani@kali:~$ gcc prio.c
ramani@kali:~$ gcc prio.c -o priol.c
ramani@kali:~$ ./prio.c
zsh: permission denied: ./prio.c
ramani@kali:~$
Enter the number of processes: 4
Enter process name: p1
Enter burst time for p1: 6
Enter priority for p1: 4
Enter process name: p2
Enter burst time for p2: 2
Enter priority for p2: 3
Enter process name: p3
Enter burst time for p3: 8
Enter priority for p3: 4
Enter process name: p4
Enter burst time for p4: 4
Enter priority for p4: 4
Process Burst Time
p1      6
p2      2
p3      8
Total Waiting Time: 14
Average Waiting Time: 4.67
Total Turnaround Time: 32
Average Turnaround Time: 10.67
ramani@kali:~$

```

6)d)

ROUND ROBIN SCHEDULING

Aim:

To implement the Round Robin (RR) scheduling technique

Example:

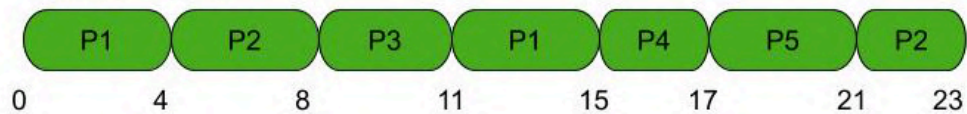
TIME SLICE = 4

PROCESS	ARRIVAL TIME	BURST TIME	
		TOTAL	REMAINING
P1	0	8	8
P2	1	6	6
P3	3	3	3
P4	5	2	2
P5	6	4	4

READY QUEUE:

P1	P2	P3	P1	P4	P5	P2
----	----	----	----	----	----	----

GANTT CHART:



Code explanation;

```

struct Process {
    char name[10];
    int arrivalTime;
    int burstTime;
    int remainingTime;
    int waitingTime;
    int turnaroundTime;
};

```

Purpose: This structure defines the properties of a process.

Use:

- **name**: For identifying the process (e.g., "P1", "P2").
- **arrivalTime**: When the process arrives in the ready queue.
- **burstTime**: How much CPU time it requires.
- **remainingTime**: Keeps track of how much time is still needed (used during scheduling).
- **waitingTime**: Final waiting time, used in output and average calculations.
- **turnaroundTime**: Final turnaround time, used for reporting and averages.

```
for (int i = 0; i < n; i++) { ... }
```

- Go through **every process** to check if it can run.

💡 **if (p[i].remainingTime > 0 && p[i].arrivalTime <= time)**

Two conditions must be true for a process to get CPU:

1. It still has some time left (**remainingTime > 0**).
2. It has already **arrived** (**arrivalTime <= time**).

If both are true, the process is **eligible to run** right now.

🔄 **If it's eligible:**

✅ **done = 0;**

- This tells the system: **"We still have work to do"**, don't exit the loop yet.

🧩 **Now we apply Round Robin logic:**

♦ If it needs more than the time quantum:

```
if (p[i].remainingTime > timeQuantum) {
    time += timeQuantum;
    p[i].remainingTime -= timeQuantum;
}
```

- Run it for `timeQuantum` (like 2 units).
- Subtract that quantum from its `remainingTime`.
- Increase the **global time** by that quantum.

➡ Imagine: "Hey Process P1, you've used 2 out of your 5 units, go wait in line again."

♦ If it needs **less than or equal to the time quantum**:

```
else {
    time += p[i].remainingTime;
    p[i].waitingTime = time - p[i].burstTime - p[i].arrivalTime;
    p[i].remainingTime = 0;
}
```

Give it **just enough time** to finish.

Advance the clock by that time.

Calculate **waiting time**:

`waitingTime = time (now) - burstTime - arrivalTime`

➤ Total time in system - time it needed to run - when it arrived = time it spent waiting.

Set `remainingTime = 0`, meaning this process is **done**.

Code:

```
#include <stdio.h>
#include <string.h>

struct Process {
    char name[10];
    int arrivalTime;
    int burstTime;
    int remainingTime;
    int waitingTime;
    int turnaroundTime;
};

int main() {
    int n, timeQuantum, time = 0, done;
    struct Process p[10];

    printf("Enter the number of processes: ");
    scanf("%d", &n);

    printf("Enter the Time Quantum: ");
    scanf("%d", &timeQuantum);

    // Step 3: Read process info
    for (int i = 0; i < n; i++) {
        printf("\nEnter process name: ");
        scanf("%s", p[i].name);
        printf("Enter arrival time for %s: ", p[i].name);
        scanf("%d", &p[i].arrivalTime);
        printf("Enter burst time for %s: ", p[i].name);
        scanf("%d", &p[i].burstTime);
        p[i].remainingTime = p[i].burstTime;
        p[i].waitingTime = 0;
    }
```



```

// Step 6 & 7: Start Round Robin
do {
    done = 1;
    for (int i = 0; i < n; i++) {
        if (p[i].remainingTime > 0 && p[i].arrivalTime <= time) {
            done = 0;
            if (p[i].remainingTime > timeQuantum) {
                time += timeQuantum;
                p[i].remainingTime -= timeQuantum;
            } else {
                time += p[i].remainingTime;
                p[i].waitingTime = time - p[i].burstTime - p[i].arrivalTime;
                p[i].remainingTime = 0;
            }
        }
    }
} while (!done);

// Step 8 & 9: Calculate turnaround time and averages
float totalWT = 0, totalTAT = 0;

printf("\nProcess\tAT\tBT\tWT\tTAT\n");
for (int i = 0; i < n; i++) {
    p[i].turnaroundTime = p[i].waitingTime + p[i].burstTime;
    totalWT += p[i].waitingTime;
    totalTAT += p[i].turnaroundTime;

    printf("%s\t%d\t%d\t%d\t%d\n", p[i].name, p[i].arrivalTime, p[i].burstTime,
p[i].waitingTime, p[i].turnaroundTime);
}

printf("\nAverage Waiting Time: %.2f", totalWT / n);
printf("\nAverage Turnaround Time: %.2f\n", totalTAT / n);

return 0;
}

```

```

ramani@kali:~$ gcc roundRobin.c
ramani@kali:~$ ./roundRobin
Enter the number of processes: 3
Enter the Time Quantum: 2

Enter process name: p1
Enter arrival time for p1: 0
Enter burst time for p1: 5

Enter process name: p2
Enter arrival time for p2: 1
Enter burst time for p2: 3

Enter process name: p3
Enter arrival time for p3: 2
Enter burst time for p3: 1

Process AT   BT   WT   TAT
p1          0   5   4   9
p2          1   3   4   7
p3          2   1   2   3

Average Waiting Time: 3.33
Average Turnaround Time: 6.33
ramani@kali:~$

```

7)

IPC USING SHARED MEMORY

Aim:

To write a C program to do Inter Process Communication (IPC) using shared memory between sender process and receiver process.

Explanation:

What is IPC (Inter-Process Communication)?

IPC stands for **Inter-Process Communication**. It refers to mechanisms that allow processes (programs running in memory) to communicate with each other.

Think of it like two apps or programs that want to **share data, send messages, or work together** — IPC makes that possible.

Types of ipc mechanism:

IPC Method	Description	Example Use
Shared Memory	Memory is shared between processes. Fastest, but needs coordination.	One process writes to memory, another reads it.
Message Queues	A process places messages in a queue, another reads them.	Log collection, messaging services.
Pipes	Data flows in one direction between two related processes.	Output from one process becomes input to another.
Named Pipes (FIFOs)	Like pipes, but works between unrelated processes.	System-wide logging or communication.
Sockets	Allows communication over a network or locally.	Chat apps, web servers.
Signals	Simple notifications (not data).	Telling a process to stop, pause, or refresh.
Semaphores	Used to manage access to shared resources (not for data transfer).	Preventing race conditions in shared memory.

What is **shmget()**?

➤ **shmget()** is a system call used to create or access a shared memory segment.

Syntax:

```
int shmget(key_t key, size_t size, int shmflg);
```

1. `key_t` is a type defined in system headers — usually in `<sys/types.h>`. It's used to **store the IPC key** generated by `ftok()`.
2. Key is the variable name to store the ipc key.

✓ `ftok("shmfile", 65)` is used to generate a **reliable, unique, and consistent key**.

⚙️ What does `ftok("shmfile", 65)` actually do?

Ftok stands for “file to key”.

Shmfile is just a random file for key generation.

1. Looks up `"shmfile"` on disk.
2. Reads the file's **inode number** (a unique ID for the file).
3. Reads the file's **device ID** (to identify the disk it's on).
4. Combines those with the number `65` (your project ID).
5. Outputs a **key** that should be **unique but consistent**.

Without a file, the OS would have no consistent way to generate a unique, predictable key that works across processes. Because:

The operating system needs a **stable, unique, and consistent reference** that:

- Exists across different processes.
- Can be used to compute a repeatable key value.

A file's **inode number + device ID** are:

- **Unique on the system**
- **Persist even after a reboot**
- **Accessible by multiple processes**

```
int shmid = shmget(key, size, 0666 | IPC_CREAT);
```

This line is creating or accessing a shared memory segment using a unique key and some flags.

shmid - The shared memory ID returned by the system. You'll use this to access the segment.

```
shmget(key_t key, size_t size, int shmflg);
```

It's a system call used to:

Create or **access** a shared memory segment in Linux (System V IPC).

key - Unique identifier (generated using `ftok()`)

size - Number of bytes of memory you want to allocate

shmflg - Flags for permissions and creation behavior

ipc_creat - If segment doesn't exist yet, **create** it.

0666 - rw-rw-rw- - all users can read & write this memory.



What it does:

1. **Searches** for a shared memory segment with that **key**.
2. If found:
 - Returns its ID (if permissions allow).
3. If not found and **IPC_CREAT** is set:
 - **Creates** a new memory segment of **size** bytes.
 - Returns the new segment ID.
4. If not found and **IPC_CREAT** is **not** set:
 - Returns **-1** (error).



What does `shmat()` do?

It **attaches** the shared memory segment to your process's address space — basically "mounts" the memory so your program can use it like normal RAM.

`char *str` - You're creating a pointer that will point to the start of the shared memory.

`(*str)` - Typecast — because `shmat()` returns a `void *` (generic pointer), we cast it to a `char *` (so we can use it for strings).

`shmat` - Function to **attach** the shared memory segment to your process.

`shmid` - The ID of the shared memory (you got this from `shmget()` earlier).

`Null` - Let the system choose the memory address to attach it at (simpler and safer).

`0` - No special flags (default behavior).

`sprintf()` **writes formatted data into a string**, instead of printing it to the screen.

It works **just like** `printf()`, but instead of sending output to the terminal, it **stores it in a character array**.

`sleep(5)`

The `sleep(5)` function **only pauses the sender process**. After that:

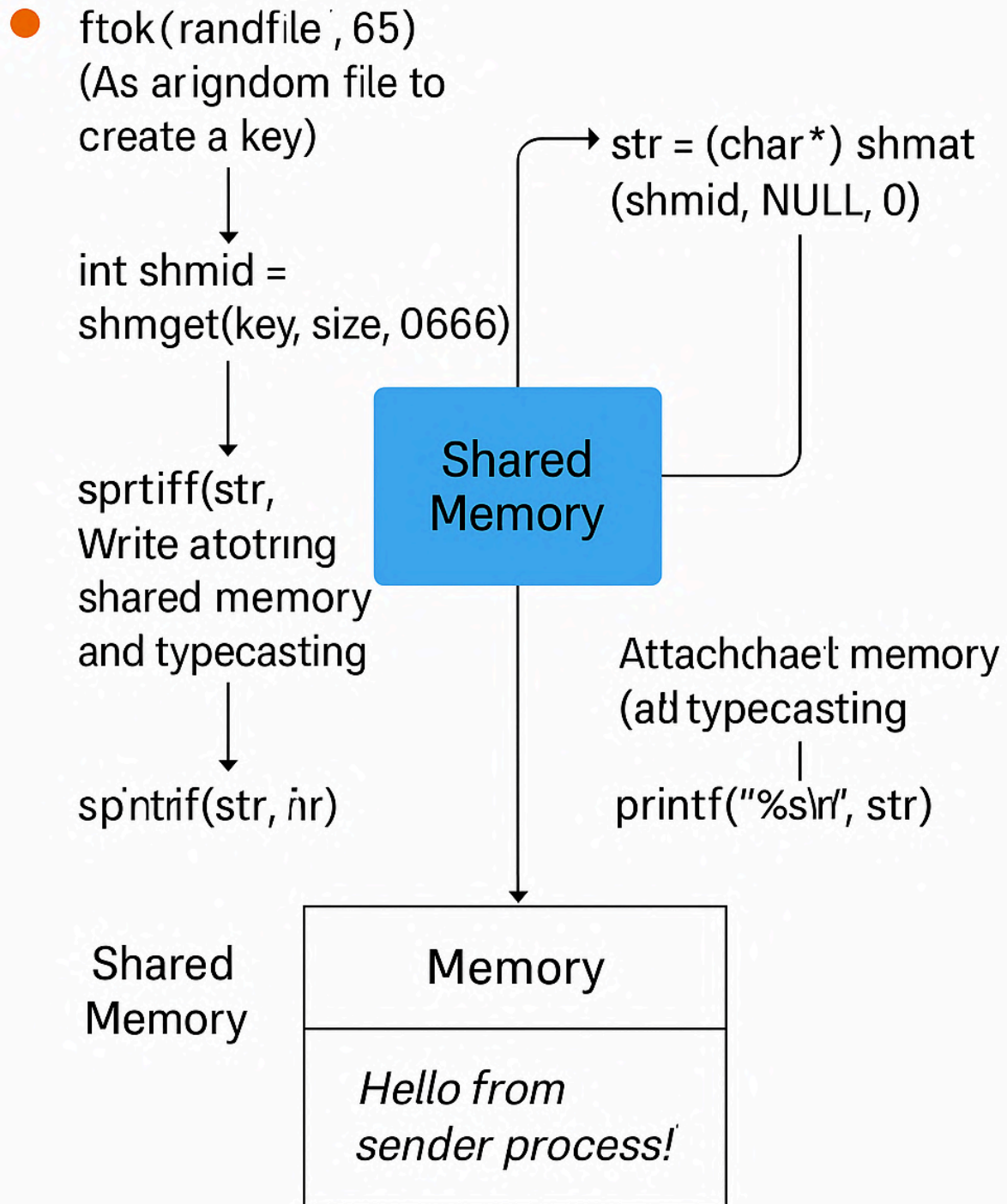
- The sender process **detaches** from the shared memory using `shmdt()`.
- But here's the key part: **detaching doesn't delete the shared memory**. It simply means the sender no longer has access to that shared memory.

`shmdt()`

The function `shmdt()` is used to detach a process from a shared memory segment after it has finished using it.

Sender Process

Receiver Process



Sender Process

Receiver Process

Code:
Sender.c

```
// sender.c
#include <stdio.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#include <string.h>
#include <unistd.h>

int main() {
    // Step 1: Set the size
    int size = 1024;

    // Step 2: Create shared memory segment
    key_t key = ftok("shmfile", 65); // Generate unique key
    int shmid = shmget(key, size, 0666 | IPC_CREAT);

    // Step 3: Attach shared memory
    char *str = (char *) shmat(shmid, NULL, 0);

    // Step 4: Write message
    sprintf(str, "Hello from sender process!");

    // Step 5: Sleep to simulate delay
    printf("Data written to shared memory: %s\n", str);
    sleep(5);

    // Step 6: Detach
    shmdt(str);

    return 0;
}
```

Receiver
Explanation:

1. **Attach to the shared memory:** The receiver process uses `shmat()` to attach to the same shared memory segment that the sender wrote to. This step allows the receiver to access and manipulate the data in the shared memory.

2. str is now used to printf the message delivered by the sender.

Receiver.c:

```
#include <stdio.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#include <unistd.h>

int main() {
    // Step 1: Set the size
    int size = 1024;

    // Step 2: Access existing shared memory
    key_t key = ftok("shmfile", 65); // Use same key as sender
    int shmid = shmget(key, size, 0666);

    // Step 3: Attach
    char *str = (char *) shmat(shmid, NULL, 0);

    // Step 4: Print received data
    printf("Received data from shared memory: %s\n", str);

    // Step 5: Detach
    shmdt(str);

    return 0;
}
```

Run both code on two separate terminal

```
File Actions Edit View Help
// sender.c
#include <stdio.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#include <unistd.h>

int main() {
    // Step 1: Set the size
    int size = 1024;

    key_t key = ftok("shmfile", 65); // Generate unique key
    int shmid = shmget(key, size, 0666 | IPC_CREAT);

    // Step 2: Attach shared memory
    char *str = (char *) shmat(shmid, NULL, 0);

    // Step 3: Write message
    sprintf(str, "Hello from sender process!");

    // Step 4: Sleep to simulate delay
    printf("Data written to shared memory: %s\n", str);
    sleep(5);

    // Step 5: Detach
    shmdt(str);

    return 0;
}

$ vi sender.c
$ ./sender.c
Data written to shared memory: Hello from sender process

File Actions Edit View Help
// receiver.c
#include <stdio.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#include <unistd.h>

int main() {
    // Step 1: Set the size
    int size = 1024;

    // Step 2: Access existing shared memory
    key_t key = ftok("shmfile", 65); // Use same key as sender
    int shmid = shmget(key, size, 0666);

    // Step 3: Attach
    char *str = (char *) shmat(shmid, NULL, 0);

    // Step 4: Print received data
    printf("Received data from shared memory: %s\n", str);

    // Step 5: Detach
    shmdt(str);

    return 0;
}

$ vi receiver.c
$ ./receiver.c
Received data from shared memory: Hello from sender process!
```

8)

PRODUCER CONSUMER USING SEMAPHORES

Aim:

To write a program to implement solution to producer consumer problem using semaphores.

Key Differences: IPC Shared Memory vs Producer-Consumer with Semaphores

Comparison: IPC Shared Memory vs Producer-Consumer with Semaphores

Feature/Aspect	IPC Shared Memory	Producer-Consumer with Semaphores
Context	Between different processes	Between threads in same process
Memory Sharing Scope	Inter-process	Intra-process
Memory Setup	ftok(), shmget(), shmat()	Global/shared variables
Complexity	Higher	Lower
Needs Semaphores?	Yes	Yes
Semaphores Used For	Process-level coordination	Thread-level coordination
Speed	Slower (more overhead)	Faster (shared address space)
Common Use Case	OS-level process communication	Thread-based buffer access

Key things to know before coding:

Feature/Aspect	Mutex	Semaphore
Definition	A Mutex (Mutual Exclusion) is a synchronization object used to enforce mutual exclusion. It allows only one thread to access a shared resource at a time.	A Semaphore is a synchronization object that controls access to a resource by multiple threads. It can be used to limit the number of threads that can access a

		particular resource simultaneously.
Ownership	A mutex has ownership – only the thread that locks it can unlock it.	A semaphore doesn't have ownership – any thread can signal (increment) or wait (decrement) it.
Purpose	Used for ensuring that only one thread accesses a critical section at a time.	Used for controlling access to a resource pool or counting available resources (e.g., available slots in a buffer).
Operations	- <code>pthread_mutex_lock()</code>: Lock the mutex. - <code>pthread_mutex_unlock()</code>: Unlock the mutex.	- <code>sem_wait()</code>: Decrements the semaphore. - <code>sem_post()</code>: Increments the semaphore.
Used For	Exclusive access to a shared resource (one at a time).	Managing resource availability (e.g., limiting threads accessing a shared buffer).

The Problem

Imagine a system with:

- **One or more producers:** they create data or "items"
- **One or more consumers:** they use (consume) that data
- **A shared fixed-size buffer:** where producers place items and consumers take them

Now here's the catch:

- **The producer must wait if the buffer is full** (it can't overwrite existing data)

- **The consumer must wait if the buffer is empty** (it can't consume something that doesn't exist)

So, we need to ensure:

- Producers **don't overwrite** data before it's consumed.
- Consumers **don't read** garbage or already-consumed data.
- Multiple threads **don't corrupt** the buffer by accessing it simultaneously.

This is a **synchronization** issue!

Code explanation of semaphores:

```
// Shared buffer
int buffer[SIZE];
int count = 0; // Tracks number of items
int in = 0;    // Index where producer adds
int out = 0;   // Index where consumer removes

// Semaphore and mutex declarations
sem_t empty;    // Tracks empty slots in buffer
sem_t full;     // Tracks filled slots in buffer
pthread_mutex_t mutex; // Ensures mutual exclusion while accessing buffer
```

 **Analogy: A Warehouse with Bins and Workers**

Imagine a small **warehouse** with 3 **storage bins** (`buffer[SIZE] = 3`).
There are:

- **Producer workers**: who deliver packages to the warehouse.
- **Consumer workers**: who pick up packages from the warehouse.
- A **warehouse manager**: who ensures that only one worker (producer or consumer) handles the bins at a time.

Now, here's how each variable fits in this analogy:

Variable-wise Analogy

Buffer[size] – The **3 bins** in the warehouse to store packages

Count - A **counter** that shows how many bins are currently filled

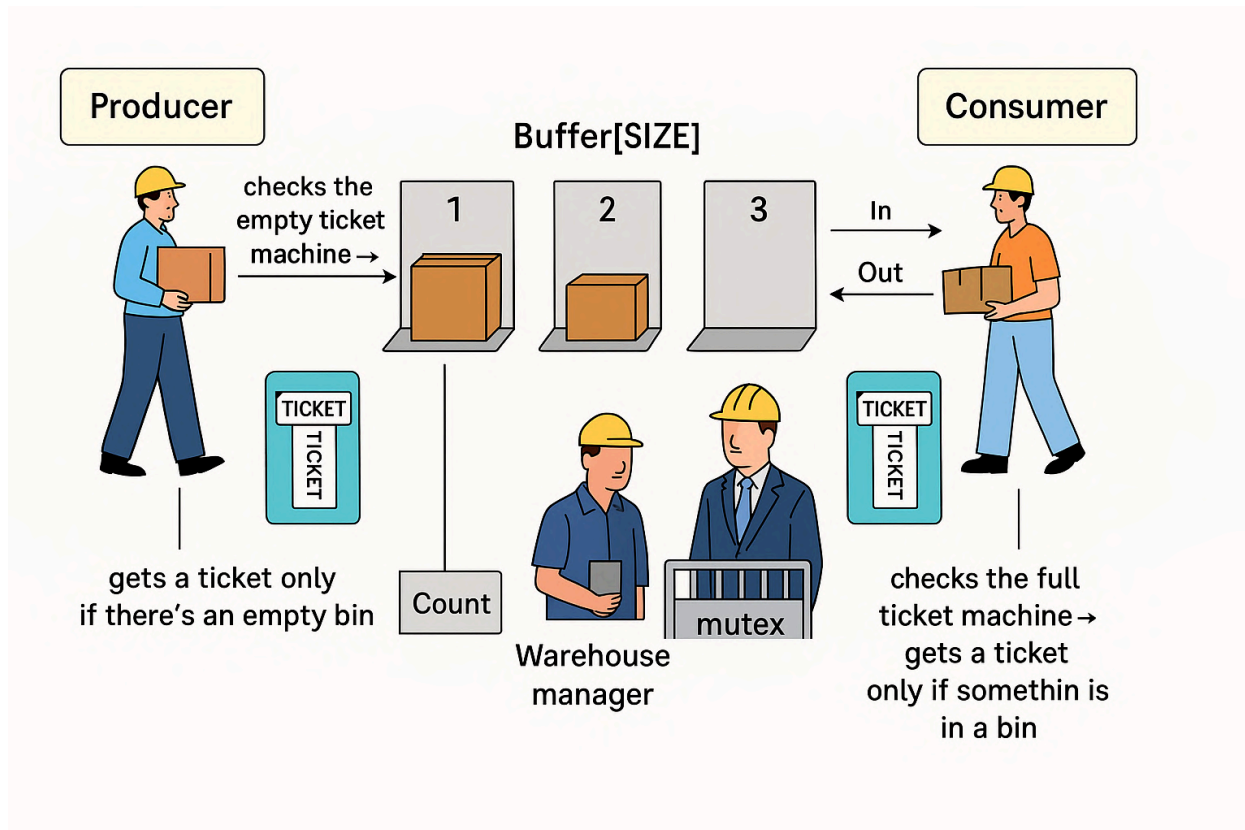
in - The **next bin index** where the producer should place a new package

Out - The **next bin index** from where the consumer should take a package

Sem_empty - A **ticket machine** giving out tickets to producers when **a bin is empty** (initially 3)

Sem_t full - A **ticket machine** giving tickets to consumers when **a bin has something to pick up** (initially 0)

pthread_mutex_t mutex - The **warehouse manager** who ensures only **one worker** is inside handling bins at any given time (so they don't bump into each other or mess up bin contents)



How It All Works Together

- A **producer** checks the **empty** ticket machine → gets a ticket only if there's an empty bin.
- Then, it asks the **warehouse manager** (mutex) to open the gate → adds a package → updates **in**, **count** → leaves and gives a **full ticket**.
- A **consumer** checks the **full** ticket machine → gets a ticket only if something is in a bin.
- Then, it asks the **manager** to go in → picks a package → updates **out**, **count** → leaves and gives back an **empty ticket**.

```
void produce() {
```

```

// Try to acquire an empty slot
if (sem_trywait(&empty) == 0) {
    pthread_mutex_lock(&mutex); // Lock the buffer

    int item = in + 1;           // Create item (sequential for demo)
    buffer[in] = item;           // Add item to buffer
    in = (in + 1) % SIZE;        // Circular increment
    count++;

    printf("Producer produces the item %d\n", item);

    pthread_mutex_unlock(&mutex); // Unlock the buffer
    sem_post(&full);             // Signal that one more slot is full
} else {
    printf("Buffer is full!!\n"); // All slots are full
}
}

```

sem_trywait(&empty) tries to decrement the **empty** semaphore. It checks if there's an empty slot available in the buffer.

If the **empty** semaphore is greater than 0 (i.e., there is at least one empty slot), it returns 0, and the producer proceeds.

If no empty slots are available (i.e., the semaphore is 0), it doesn't block the thread and just prints **"Buffer is full!!"**.

```
pthread_mutex_lock(&mutex); // Lock the buffer
```

- This line ensures that only one thread (producer or consumer) can access the **shared buffer** at a time. It's used to prevent multiple threads from simultaneously modifying the buffer, which would lead to race conditions and data corruption.

item = in + 1; The producer creates an item. Here, it's just a sequential number (**in + 1**).

buffer[in] = item; The item is added to the buffer at the index **in**.

in = (in + 1) % SIZE; This makes the **in** pointer circular. When **in** reaches **SIZE**, it wraps around back to 0.

count++: Increments the count of items in the buffer.

`pthread_mutex_unlock(&mutex);` // Unlock the buffer

1. After producing and modifying the buffer, the mutex is **unlocked**, allowing other threads (such as consumers) to access the buffer.

Signal Full Slot (sem_post(&full)**):**

1. **sem_post(&full)** increments the **full** semaphore to signal that there is one more item in the buffer. This notifies the consumer that there's now something to consume.

```
else {
    printf("Buffer is full!!\n"); // All slots are full
}
```

1. If there are no empty slots (**sem_trywait(&empty)** returned a non-zero value), the producer prints **"Buffer is full!!"** and does not proceed to produce an item.

Consumer program explanation;

// Function to consume item


```

void consume() {
    // Try to acquire a full slot
    if (sem_trywait(&full) == 0) {
        pthread_mutex_lock(&mutex); // Lock the buffer

        int item = buffer[out]; // Remove item from buffer
        buffer[out] = 0;        // Clear the slot (for clarity)
        out = (out + 1) % SIZE;  // Circular increment
        count--;

        printf("Consumer consumes item\n%d\n", item);

        pthread_mutex_unlock(&mutex); // Unlock the buffer
        sem_post(&empty);             // Signal that one more slot is empty
    } else {
        printf("Buffer is empty!!\n"); // All slots are empty
    }
}

```

```

if (sem_trywait(&full) == 0) {

```

1. `sem_trywait(&full)`: This checks if there is at least one full slot in the buffer.

2. If yes (`value > 0`): it decrements the full semaphore and proceeds.

2. If no (buffer is empty): it skips the body and prints "Buffer is empty!!".

Using `trywait` means it won't block if the buffer is empty — it just fails immediately.

`pthread_mutex_lock(&mutex);` // Lock the buffer

1. Locks the **mutex** so that no other thread (producer or consumer) can modify the buffer simultaneously.
2. Ensures **mutual exclusion** for safe access to the buffer.

Consume item:

```
int item = buffer[out]; // Remove item from buffer
buffer[out] = 0;        // Clear the slot (for clarity)
out = (out + 1) % SIZE;  // Circular increment
count--;
```

`item = buffer[out];` Retrieves the item at the **current consumer index**.

`buffer[out] = 0;` Clears the slot — optional, for visual clarity/debugging.

`out = (out + 1) % SIZE;` Advances the `out` pointer **circularly**, i.e., loops back to 0 when it reaches SIZE.

`count--;` Decreases the number of items in the buffer.

Display consumed item

```
printf("Consumer consumes item\n%d\n", item);
```

Prints the item that was just consumed.

Exit critical section

```
pthread_mutex_unlock(&mutex);
```

 // Unlock the buffer

Releases the mutex so that other threads (like a producer) can access the buffer.

Signal availability

```
sem_post(&empty);
```

 // Signal that one more slot is empty

1. Increments the **empty** semaphore since one slot in the buffer is now free.
2. This notifies the producer that it can now add another item.

✗ If buffer was empty

```
} else {
    printf("Buffer is empty!!\n");
}
```

Code;

```
#include <stdio.h>
#include <stdlib.h>
#include <semaphore.h>
#include <pthread.h>

#define SIZE 3 // Buffer size

// Shared buffer
int buffer[SIZE];
int count = 0; // Tracks number of items
int in = 0; // Index where producer adds
int out = 0; // Index where consumer removes

// Semaphore and mutex declarations
sem_t empty; // Tracks empty slots in buffer
sem_t full; // Tracks filled slots in buffer
pthread_mutex_t mutex; // Ensures mutual exclusion while accessing buffer
```

// Function to produce item

```
void produce() {
    // Try to acquire an empty slot
    if (sem_trywait(&empty) == 0) {
        pthread_mutex_lock(&mutex); // Lock the buffer

        int item = in + 1;          // Create item (sequential for demo)
        buffer[in] = item;          // Add item to buffer
        in = (in + 1) % SIZE;       // Circular increment
        count++;

        printf("Producer produces the item %d\n", item);

        pthread_mutex_unlock(&mutex); // Unlock the buffer
        sem_post(&full);             // Signal that one more slot is full
    } else {
        printf("Buffer is full!!\n"); // All slots are full
    }
}
```

// Function to consume item

```
void consume() {
    // Try to acquire a full slot
    if (sem_trywait(&full) == 0) {
        pthread_mutex_lock(&mutex); // Lock the buffer

        int item = buffer[out]; // Remove item from buffer
        buffer[out] = 0;        // Clear the slot (for clarity)
        out = (out + 1) % SIZE;  // Circular increment
        count--;

        printf("Consumer consumes item\n%d\n", item);

        pthread_mutex_unlock(&mutex); // Unlock the buffer
        sem_post(&empty);             // Signal that one more slot is empty
    }
}
```

```

    } else {
        printf("Buffer is empty!!\n"); // All slots are empty
    }
}

int main() {
    int choice;

    // Initialize semaphores
    sem_init(&empty, 0, SIZE); // Initially all slots are empty
    sem_init(&full, 0, 0);    // No full slots initially
    pthread_mutex_init(&mutex, NULL); // Initialize mutex

    // Menu loop
    while (1) {
        printf("\n1. Producer\n2. Consumer\n3. Exit\nEnter your choice:");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                produce(); // Call produce function
                break;
            case 2:
                consume(); // Call consume function
                break;
            case 3:
                // Cleanup before exit
                printf("Exiting...\n");
                sem_destroy(&empty);
                sem_destroy(&full);
                pthread_mutex_destroy(&mutex);
                exit(0);
            default:
                printf("Invalid choice!\n"); // Handle wrong input
        }
    }
}

```

```

    }

    return 0;
}

```

```

#include <stdio.h>
#include <stdlib.h>
#include <semaphore.h>
#include <pthread.h>

#define SIZE 3

int buffer[SIZE];
int count = 0;
int in = 0;
int out = 0;

sem_t empty;
sem_t full;
pthread_mutex_t mutex;

void produce() {
    if (sem_trywait(&empty) == 0) {
        pthread_mutex_lock(&mutex);
        int item = in + 1;
        buffer[in] = item;
        in = (in + 1) % SIZE;
        count++;
        printf("Producer produces the item %d\n", item);
        pthread_mutex_unlock(&mutex);
        sem_post(&full);
    } else {
        printf("Buffer is full!!\n");
    }
}

void consume() {
    if (sem_trywait(&full) == 0) {
        pthread_mutex_lock(&mutex);
        int item = buffer[out];
        buffer[out] = 0;
        out = (out + 1) % SIZE;
        count--;
        printf("Consumer consumes item %d\n", item);
        pthread_mutex_unlock(&mutex);
        sem_post(&empty);
    } else {
        printf("Buffer is empty!!\n");
    }
}

int main() {
    int choice;

    sem_init(&empty, 0, SIZE);
    sem_init(&full, 0, 0);
    pthread_mutex_init(&mutex, NULL);

    while (1) {
        printf("\n1. Producer\n2. Consumer\n3. Exit\nEnter your choice:");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                produce();
                break;
            case 2:
                consume();
                break;
            case 3:
                printf("Exiting...\n");
                sem_destroy(&empty);
                sem_destroy(&full);
                pthread_mutex_destroy(&mutex);
                exit(0);
            default:
                printf("Invalid choice!\n");
        }
    }

    return 0;
}

```

```

ramani@kali:~$ ./semaphores1.c
1. Producer
2. Consumer
3. Exit
Enter your choice:1
Producer produces the item 1
1. Producer
2. Consumer
3. Exit
Enter your choice:2
Consumer consumes item 1
1. Producer
2. Consumer
3. Exit
Enter your choice:1
Producer produces the item 2
1. Producer
2. Consumer
3. Exit
Enter your choice:2
Consumer consumes item 2
1. Producer
2. Consumer
3. Exit
Enter your choice:1
Buffer is empty!!
1. Producer
2. Consumer
3. Exit
Enter your choice:3
Exiting...
ramani@kali:~$

```

DEADLOCK AVOIDANCE

Aim:

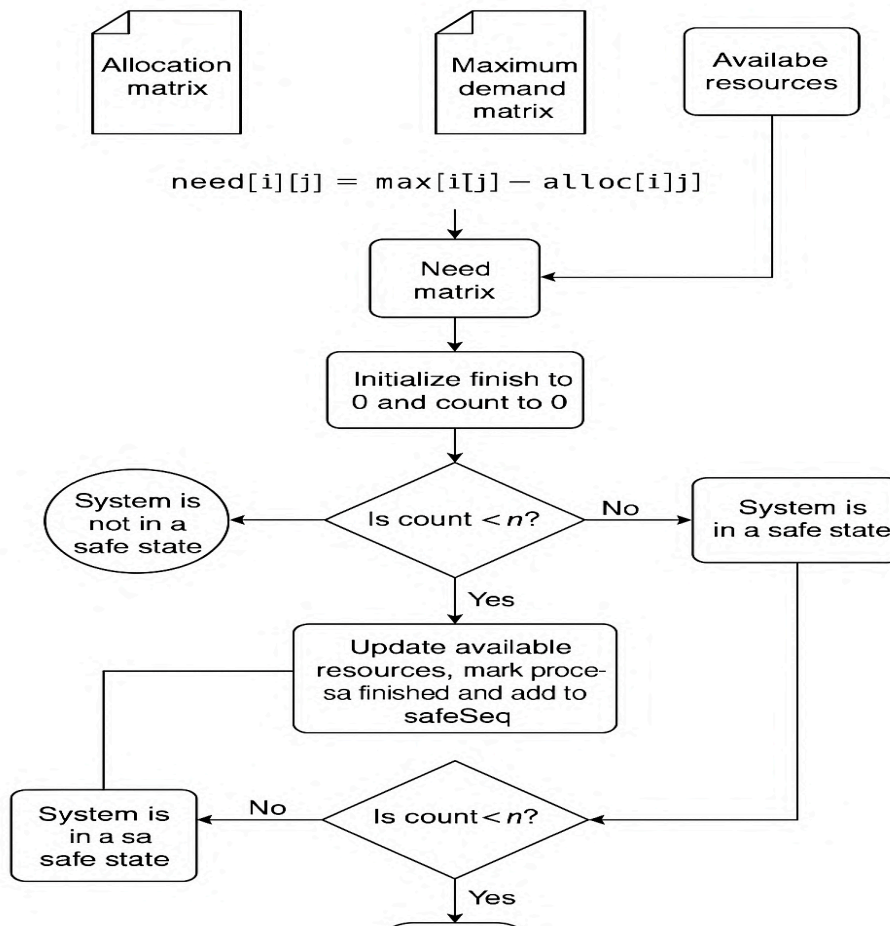
To find out a safe sequence using Banker's algorithm for deadlock avoidance.

Algorithm:

1. Initialize work=available and finish[i]=false for all values of i
2. Find an i such that both:

- $finish[i]=false$ and
 $Need_i \leq work$ 3. If no such i exists go to step 6
 4. Compute $work=work+allocation_i$
 5. Assign $finish[i]$ to true and go to step 2
 6. If $finish[i]==true$ for all i , then print safe sequence
 7. Else print there is no safe sequence

Explanation;



Code explanation;

Variable used ;

-
- n : An integer that stores the number of processes in the system.
 - m : An integer that represents the number of different resource types.

- i, j, k : Integer loop variables used to iterate through arrays and matrices.
- $alloc[MAX][MAX]$: A 2D array (matrix) that stores how many resources of each type have been **currently allocated** to each process.
For example, $alloc[1][2]$ is how many units of resource type 2 are allocated to process 1.
- $max[MAX][MAX]$: A 2D array representing the **maximum resources** that each process could potentially request.
 $max[i][j]$ is the maximum demand of process i for resource type j .
- $need[MAX][MAX]$: A 2D array that calculates and stores how many more resources each process still needs to complete its task.
It is computed as $need[i][j] = max[i][j] - alloc[i][j]$.
- $avail[MAX]$: A 1D array that stores how many units of each resource type are currently **available** in the system.
For example, $avail[2]$ is the number of available units of resource type 2.
- $finish[MAX]$: A 1D array used to mark whether a process has finished execution or not.
 $finish[i] = 1$ means process i is finished, and 0 means it is still pending.
- $safeSeq[MAX]$: A 1D array that stores the **safe sequence** of process execution — i.e., the order in which processes can complete without causing a deadlock.
- $count$: An integer counter that keeps track of how many processes have safely finished so far during the Banker's Algorithm execution.

1. Input Phase

- User inputs:
 - n : Number of processes
 - m : Number of resource types
 - Matrices and arrays read from input:
 - $alloc[n][m]$: Number of resources currently allocated to each process
 - $max[n][m]$: Maximum resources each process may need
 - $avail[m]$: Available units of each resource type
-

2. Need Matrix Calculation

- Formula:

Program Code:

```
#include <stdio.h>
#define MAX 10

int main() {
    int n, m, i, j, k;
    int alloc[MAX][MAX], max[MAX][MAX],
    need[MAX][MAX], avail[MAX];
    int finish[MAX] = {0}, safeSeq[MAX];
    int count = 0;

    printf("Enter the number of processes: ");
    scanf("%d", &n);

    printf("Enter the number of resource types:
");
    scanf("%d", &m);

    printf("\nEnter the allocation matrix:\n");
    for (i = 0; i < n; i++) {
        printf("Process %d: ", i);
        for (j = 0; j < m; j++) {
            scanf("%d", &alloc[i][j]);
        }
    }

    printf("\nEnter the maximum matrix:\n");
    for (i = 0; i < n; i++) {
        printf("Process %d: ", i);
        for (j = 0; j < m; j++) {
            scanf("%d", &max[i][j]);
        }
    }
}
```

```

printf("\nEnter the available resources:\n");
for (i = 0; i < m; i++) {
    scanf("%d", &avail[i]);
}

// Calculate the need matrix
for (i = 0; i < n; i++) {
    for (j = 0; j < m; j++) {
        need[i][j] = max[i][j] - alloc[i][j];
    }
}

// Banker's algorithm
while (count < n) {
    int found = 0;
    for (i = 0; i < n; i++) {
        if (!finish[i]) {
            int canAllocate = 1;
            for (j = 0; j < m; j++) {
                if (need[i][j] > avail[j]) {
                    canAllocate = 0;
                    break;
                }
            }

            if (canAllocate) {
                for (k = 0; k < m; k++) {
                    avail[k] += alloc[i][k];
                }
                safeSeq[count++] = i;
                finish[i] = 1;
                found = 1;
            }
        }
    }

    if (!found) {
        printf("\nSystem is not in a safe
state.\n");
        return 0;
    }
}

```

```

    printf("\nSystem is in a safe state.\nSafe
sequence is: ");
    for (i = 0; i < n; i++) {
        printf("P%d ", safeSeq[i]);
    }
    printf("\n");

    return 0;
}

```

```

#include <stdio.h>
#define MAX 10

int main() {
    int n, m, i, j, k;
    int alloc[MAX][MAX], max[MAX][MAX],
    need[MAX][MAX], avail[MAX];
    int finish[MAX] = {0}, safeSeq[MAX];
    int count = 0;

    printf("Enter the number of processes: ");
    scanf("%d", &n);

    printf("Enter the number of resource types: ");
    scanf("%d", &m);

    printf("\nEnter the allocation matrix:\n");
    for (i = 0; i < n; i++) {
        printf("Process %d: ", i);
        for (j = 0; j < m; j++) {
            scanf("%d", &alloc[i][j]);
        }
    }

    printf("\nEnter the maximum matrix:\n");
    for (i = 0; i < n; i++) {
        printf("Process %d: ", i);
        for (j = 0; j < m; j++) {
            scanf("%d", &max[i][j]);
        }
    }

    printf("\nEnter the available resources:\n");
    for (i = 0; i < m; i++) {
        scanf("%d", &avail[i]);
    }

    // Calculate the need matrix
    for (i = 0; i < n; i++) {
        for (j = 0; j < m; j++) {
            need[i][j] = max[i][j] - alloc[i][j];
        }
    }

    // Banker's algorithm
    while (count < n) {
        for (j = 0; j < m; j++) {
            need[i][j] = max[i][j] - alloc[i][j];
        }
    }

    // Banker's algorithm
    while (count < n) {
        int found = 0;
        for (i = 0; i < n; i++) {
            if (!finish[i]) {
                int canAllocate = 1;
                for (j = 0; j < m; j++) {
                    if (need[i][j] > avail[j]) {
                        canAllocate = 0;
                        break;
                    }
                }
                if (canAllocate) {
                    for (k = 0; k < m; k++) {
                        avail[k] += alloc[i][k];
                    }
                    safeSeq[count++] = i;
                    finish[i] = 1;
                    found = 1;
                }
            }
        }
        if (!found) {
            printf("\nSystem is not in a safe state.\n");
            return 0;
        }
    }

    printf("\nSystem is in a safe state.\nSafe sequence is: ");
    for (i = 0; i < n; i++) {
        printf("P%d ", safeSeq[i]);
    }
    printf("\n");

    return 0;
}

```

BEST FIT

Aim:

To implement Best Fit memory allocation technique using Python.

Algorithm:

1. Input memory blocks and processes with sizes
2. Initialize all memory blocks as free.
3. Start by picking each process and find the minimum block size that can be assigned to current process
4. If found then assign it to the current process.
5. If not found then leave that process and keep checking the further processes.

Program Code:

```

#include <stdio.h>

#define MAX 25

int main() {
    int blockSize[MAX], processSize[MAX],
    blockAllocated[MAX];
    int blockCount, processCount;
    int i, j, bestIdx;

    printf("Enter the number of memory blocks:
");
    scanf("%d", &blockCount);

    printf("Enter the size of each memory
block:\n");
    for (i = 0; i < blockCount; i++) {
        printf("Block %d: ", i + 1);
        scanf("%d", &blockSize[i]);
        blockAllocated[i] = 0; // Initially all
blocks are free
    }

    printf("\nEnter the number of processes: ");
    scanf("%d", &processCount);

    printf("Enter the size of each process:\n");
    for (i = 0; i < processCount; i++) {
        printf("Process %d: ", i + 1);
        scanf("%d", &processSize[i]);
    }

    int allocation[MAX];
    for (i = 0; i < processCount; i++) {
        bestIdx = -1;

        for (j = 0; j < blockCount; j++) {
            if (!blockAllocated[j] &&
blockSize[j] >= processSize[i]) {

```

```

        if (bestIdx == -1 || blockSize[j]
< blockSize[bestIdx]) {
            bestIdx = j;
        }
    }

    if (bestIdx != -1) {
        allocation[i] = bestIdx;
        blockAllocated[bestIdx] = 1;
    } else {
        allocation[i] = -1;
    }
}

printf("\nProcess No\tProcess Size\tBlock
No\n");
for (i = 0; i < processCount; i++) {
    printf("%d\t\t%d\t\t", i + 1,
processSize[i]);
    if (allocation[i] != -1)
        printf("%d\n", allocation[i] + 1);
    else
        printf("Not Allocated\n");
}

return 0;
}

```

```

File Actions Edit View Help
ramani@kali: ~
$ gcc bestfit.c
$ ./bestfit.c
Enter the number of memory blocks: 5
Enter the size of each memory block:
Block 1: 100
Block 2: 500
Block 3: 200
Block 4: 300
Block 5: 600
Enter the number of processes: 4
Enter the size of each process:
Process 1: 212
Process 2: 437
Process 3: 112
Process 4: 426
Process No    Process Size    Block No
1             212          4
2             437          2
3             112          3
4             426          5
ramani@kali: ~

```

FIRST FIT

Aim:

To write a C program for implementation memory allocation methods for fixed partition using first fit.

Algorithm:

1. Define the max as 25.
- 2: Declare the variable frag[max], b[max], f[max], i, j, nb, nf, temp, highest=0, bf[max], ff[max].
- 3: Get the number of blocks, files, size of the blocks using for loop.
- 4: In for loop check $bf[j] \neq 1$, if so $temp = b[j] - f[i]$
- 5: Check highest

Program Code:

```

#include <stdio.h>
#define MAX 25

int main() {
    int frag[MAX], b[MAX], f[MAX], i, j, nb, nf,
    temp;

```

```

    static int bf[MAX], ff[MAX];

    printf("Enter the number of blocks: ");
    scanf("%d", &nb);

    printf("Enter the number of files: ");
    scanf("%d", &nf);

    printf("\nEnter the size of each block:\n");
    for (i = 0; i < nb; i++) {
        printf("Block %d: ", i + 1);
        scanf("%d", &b[i]);
    }

    printf("\nEnter the size of each file:\n");
    for (i = 0; i < nf; i++) {
        printf("File %d: ", i + 1);
        scanf("%d", &f[i]);
    }

    for (i = 0; i < nf; i++) {
        for (j = 0; j < nb; j++) {
            if (bf[j] != 1 && b[j] >= f[i]) {
                ff[i] = j;
                frag[i] = b[j] - f[i];
                bf[j] = 1;
                break;
            }
        }
    }

    printf("\nFile No\tFile Size\tBlock No\tBlock
Size\tFragment\n");
    for (i = 0; i < nf; i++) {
        printf("%d\t%d\t\t%d\t\t%d\t\t%d\n",
            i + 1, f[i], ff[i] + 1, b[ff[i]],
            frag[i]);
    }

    return 0;
}

```

```

#include <stdio.h>
#define MAX 25

int main() {
    int frag[MAX], b[MAX], f[MAX], i, j, nb, nf,
    temp;
    static int bf[MAX], ff[MAX];

    printf("Enter the number of blocks: ");
    scanf("%d", &nb);

    printf("Enter the number of files: ");
    scanf("%d", &nf);

    printf("\nEnter the size of each block:\n");
    for (i = 0; i < nb; i++) {
        printf("Block %d: ", i + 1);
        scanf("%d", &b[i]);
    }

    printf("\nEnter the size of each file:\n");
    for (i = 0; i < nf; i++) {
        printf("File %d: ", i + 1);
        scanf("%d", &f[i]);
    }

    for (i = 0; i < nf; i++) {
        for (j = 0; j < nb; j++) {
            if (b[j] >= f[i]) {
                ff[i] = j;
                bf[j] = i;
                break;
            }
        }
    }

    printf("\nFile No\tFile Size\tBlock No\tBlock Size\tFragment\n");
    for (i = 0; i < nf; i++) {
        printf("%d\t%d\t%d\t%d\t%d\n",
            i + 1, f[i], ff[i] + 1, b[ff[i]], f[i]);
    }

    return 0;
}

```

```

ramani@kali:~$ vi firstfit.c
ramani@kali:~$ gcc firstfit.c -o firstfit
ramani@kali:~$ ./firstfit
Enter the number of blocks: 5
Enter the number of files: 4
Enter the size of each block:
Block 1: 100
Block 2: 500
ramani@kali:~$ ./firstfit
Enter the number of blocks: 5
Enter the number of files: 4
Enter the size of each block:
Block 1: 100
Block 2: 500
Block 3: 200
Block 4: 300
Block 5: 600
Enter the size of each file:
File 1: 212
File 2: 417
File 3: 112
File 4: 426
ramani@kali:~$

```

File No	File Size	Block No	Block Size	Fragment
1	212	2	500	288
2	417	5	600	183
3	112	3	200	88
4	426	1	100	0

FIFO PAGE REPLACEMENT

Aim:

To find out the number of page faults that occur using First-in First-out (FIFO) page replacement technique.

Algorithm:

1. Declare the size with respect to page length
2. Check the need of replacement from the page to memory
3. Check the need of replacement from old page to new page in memory 4.

Form a queue to hold all pages

5. Insert the page require memory into the queue
6. Check for bad replacement and page fault
7. Get the number of processes to be inserted
8. Display the values

Explanation:

To simulate the **FIFO (First-In-First-Out) Page Replacement Algorithm** and calculate the number of **page faults** that occur during execution.



Concept Explanation: Page Replacement and FIFO

In operating systems, when a **page** is needed (a portion of memory) but it's **not present in the main memory (RAM)**, a **page fault** occurs.

- **Page Fault:** When the required page is **not in the memory**.
- If there's **no free frame** in memory, **one of the existing pages needs to be replaced**.
- **FIFO Replacement Strategy:** Replace the **oldest loaded page** in memory (i.e., the one that was added first).



What You Need To Do

You're given:

1. A **page reference string** (like a list of page numbers that are accessed by a process, e.g., **7 0 1 2 0 3 0 4 2 3**)
2. The number of **frames** (slots in memory that can hold a page)

Your tasks:

- Simulate **how pages are loaded into frames** using FIFO.
 - Count **how many page faults** occur (every time a new page is brought in and wasn't already in memory).
 - Print the **state of the memory** after each page reference, indicating whether a page fault occurred.
-



Example

Let's say you have:

- **Page reference string:** 1 2 3 4 1 2 5 1 2 3 4 5
- **Frame count:** 3

Now simulate:

1. Insert 1 → Page fault (frame: 1)
2. Insert 2 → Page fault (frame: 1 2)
3. Insert 3 → Page fault (frame: 1 2 3)
4. Insert 4 → Page fault, replace 1 (frame: 4 2 3)
5. Insert 1 → Page fault, replace 2 (frame: 4 1 3) ... and so on.

Count total number of page faults in the end.

Program explanation:

Function definition;

```
void fifoPageReplacement(int pages[], int pageCount, int frameCount)
```

This function simulates FIFO page replacement.

pages[] – the page reference string (an array of page numbers)

pageCount – total number of pages in the reference string

frameCount – number of frames (slots in memory)

Variables inside:

```
int frame[frameCount];
int front = 0, pageFaults = 0;
```

1. `frame[]` – array that represents memory frames; it holds currently loaded pages.
2. `front` – pointer to track which page to replace next (FIFO concept: the oldest one).
3. `pageFaults` – counter to keep track of how many page faults occur.

Initialization of Memory Frames

```
for (int i = 0; i < frameCount; i++)
    frame[i] = -1;
```

Sets all frame slots to `-1`, which means "empty" initially.

Main Loop: Iterate Over Each Page in the Reference String;

```
for (int i = 0; i < pageCount; i++) {           //

    int found = 0;
    for (int j = 0; j < frameCount; j++) {
        if (frame[j] == pages[i]) {
            found = 1;
            break;
        }
    }
}
```

`found` – flag to track if the current page is already loaded in memory.

If page is already in the `frame[]`, no page fault, nothing to replace.

Program Code:

```
#include <stdio.h>

void fifoPageReplacement(int pages[], int
pageCount, int frameCount) {
    int frame[frameCount];
    int front = 0, pageFaults = 0;

    for (int i = 0; i < frameCount; i++)
        frame[i] = -1;
```

```

for (int i = 0; i < pageCount; i++) {
    int found = 0;

    // Check if page is already in frame
    for (int j = 0; j < frameCount; j++) {
        if (frame[j] == pages[i]) {
            found = 1;
            break;
        }
    }

    // If not found, it's a page fault
    if (!found) {
        frame[front] = pages[i];
        front = (front + 1) % frameCount;
        pageFaults++;
    }

    // Display frame state
    printf("Page %d: ", pages[i]);
    for (int j = 0; j < frameCount; j++) {
        if (frame[j] != -1)
            printf("%d ", frame[j]);
        else
            printf("- ");
    }
    if (!found)
        printf("(Page Fault)");
    printf("\n");
}

printf("\nTotal Page Faults = %d\n",
pageFaults);
}

int main() {
    int pageCount, frameCount;

    printf("Enter the number of pages: ");
    scanf("%d", &pageCount);
    int pages[pageCount];

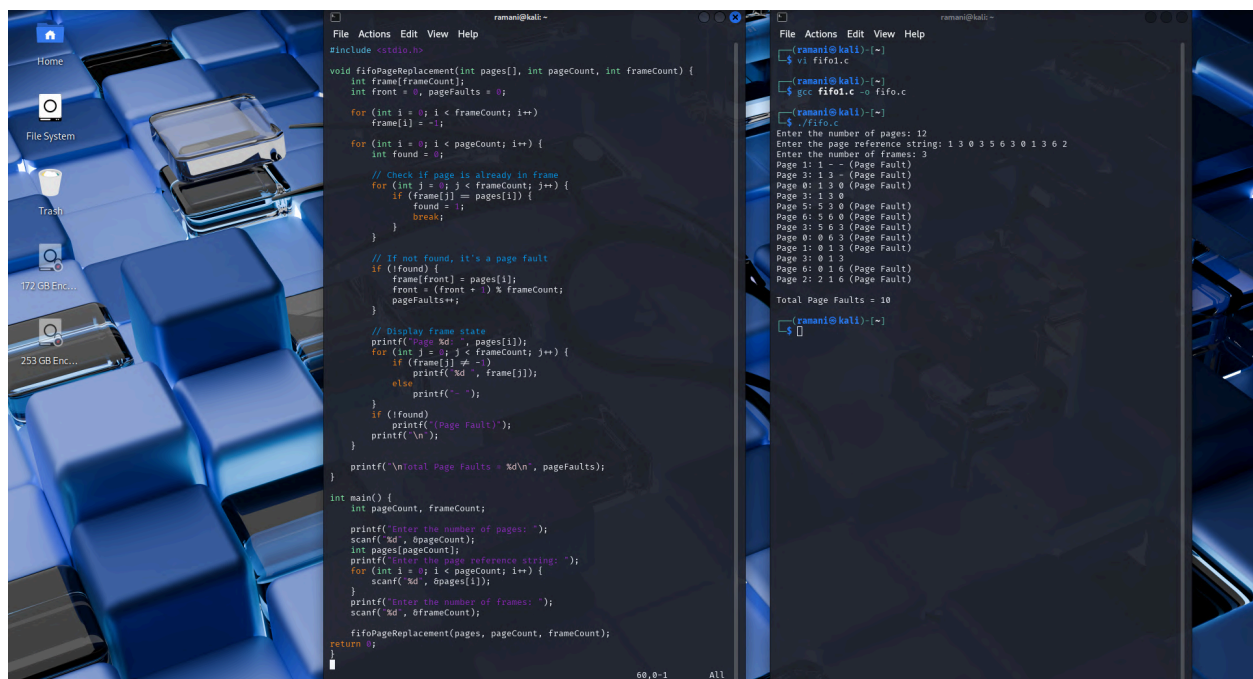
```

```

printf("Enter the page reference string: ");
for (int i = 0; i < pageCount; i++) {
    scanf("%d", &pages[i]);
}
printf("Enter the number of frames: ");
scanf("%d", &frameCount);

fifoPageReplacement(pages, pageCount,
frameCount);
return 0;
}

```



```

File Actions Edit View Help
#include <stdio.h>

void fifoPageReplacement(int pages[], int pageCount, int frameCount) {
    int frame[frameCount];
    int front = 0, pageFaults = 0;

    for (int i = 0; i < frameCount; i++)
        frame[i] = -1;

    for (int i = 0; i < pageCount; i++) {
        int found = 0;

        // Check if page is already in frame
        for (int j = 0; j < frameCount; j++) {
            if (frame[j] == pages[i]) {
                found = 1;
                break;
            }
        }

        // If not found, it's a page fault
        if (!found) {
            frame[front] = pages[i];
            front = (front + 1) % frameCount;
            pageFaults++;
        }

        // Display frame state
        printf("Page %d: ", pages[i]);
        for (int j = 0; j < frameCount; j++) {
            if (frame[j] != -1)
                printf("%d ", frame[j]);
            else
                printf("- ");
        }
        if (!found)
            printf(" (Page Fault)");
        printf("\n");
    }

    printf("\nTotal Page Faults = %d\n", pageFaults);
}

int main() {
    int pageCount, frameCount;

    printf("Enter the number of pages: ");
    scanf("%d", &pageCount);
    int pages[pageCount];
    printf("Enter the page reference string: ");
    for (int i = 0; i < pageCount; i++) {
        scanf("%d", &pages[i]);
    }
    printf("Enter the number of frames: ");
    scanf("%d", &frameCount);

    fifoPageReplacement(pages, pageCount, frameCount);
    return 0;
}

```

```

File Actions Edit View Help
(ramani@kali) ~
$ vi fifo.c
(ramani@kali) ~
$ gcc fifo.c -o fifo
(ramani@kali) ~
$ ./fifo
Enter the number of pages: 12
Enter the page reference string: 1 3 0 3 5 6 3 0 1 3 6 2
Enter the number of frames: 3
Page 1: 1 - - (Page Fault)
Page 3: 1 3 - (Page Fault)
Page 0: 1 3 0 (Page Fault)
Page 3: 1 3 0
Page 5: 5 3 0 (Page Fault)
Page 6: 5 6 0 (Page Fault)
Page 3: 5 6 3 (Page Fault)
Page 0: 0 6 3 (Page Fault)
Page 1: 0 1 3 (Page Fault)
Page 3: 0 1 3
Page 6: 0 1 6 (Page Fault)
Page 2: 2 1 6 (Page Fault)
Total Page Faults = 10
(ramani@kali) ~
$

```

LRU

Aim:

To write a c program to implement LRU page replacement algorithm.

Algorithm:

- 1: Start the process
- 2: Declare the size
- 3: Get the number of pages to be inserted
- 4: Get the value
- 5: Declare counter and stack

- 6: Select the least recently used page by counter value
- 7: Stack them according the selection.
- 8: Display the values
- 9: Stop the process

Program Code:

```
#include <stdio.h>

int findLRU(int time[], int frameCount) {
    int min = time[0], pos = 0;
    for (int i = 1; i < frameCount; i++) {
        if (time[i] < min) {
            min = time[i];
            pos = i;
        }
    }
    return pos;
}

void lruPageReplacement(int pages[], int
pageCount, int frameCount) {
    int frame[frameCount], time[frameCount];
    int pageFaults = 0, counter = 0;
    int flag1, flag2;

    for (int i = 0; i < frameCount; i++) {
        frame[i] = -1;
        time[i] = 0;
    }

    for (int i = 0; i < pageCount; i++) {
        flag1 = flag2 = 0;

        for (int j = 0; j < frameCount; j++) {
            if (frame[j] == pages[i]) {
                counter++;
                time[j] = counter;
                flag1 = flag2 = 1;
                break;
            }
        }
    }
}
```

```

        if (!flag1) {
            for (int j = 0; j < frameCount; j++)
            {
                if (frame[j] == -1) {
                    counter++;
                    frame[j] = pages[i];
                    time[j] = counter;
                    flag2 = 1;
                    pageFaults++;
                    break;
                }
            }
        }
    }

```

```

    if (!flag2) {
        int pos = findLRU(time, frameCount);
        counter++;
        frame[pos] = pages[i];
        time[pos] = counter;
        pageFaults++;
    }

```

```

    printf("Page %d: ", pages[i]);
    for (int j = 0; j < frameCount; j++) {
        if (frame[j] != -1)
            printf("%d ", frame[j]);
        else
            printf("- ");
    }
    printf("\n");
}

```

```

    printf("\nTotal Page Faults = %d\n",
pageFaults);
}

```

```

int main() {
    int pageCount, frameCount;

    printf("Enter the number of pages: ");
    scanf("%d", &pageCount);

```

```

int pages[pageCount];
printf("Enter the page reference string: ");
for (int i = 0; i < pageCount; i++) {
    scanf("%d", &pages[i]);
}

printf("Enter the number of frames: ");
scanf("%d", &frameCount);

lruPageReplacement(pages, pageCount,
frameCount);

return 0;
}

```

```

#include <stdio.h>

int findLRU(int time[], int frameCount) {
    int min = time[0], pos = 0;
    for (int i = 1; i < frameCount; i++) {
        if (time[i] < min) {
            min = time[i];
            pos = i;
        }
    }
    return pos;
}

void lruPageReplacement(int pages[], int pageCount, int frameCount) {
    int frame[frameCount], time[frameCount];
    int pageFaults = 0, counter = 0;
    int flag1, flag2;

    for (int i = 0; i < frameCount; i++) {
        frame[i] = -1;
        time[i] = 0;
    }

    for (int i = 0; i < pageCount; i++) {
        flag1 = flag2 = 0;

        for (int j = 0; j < frameCount; j++) {
            if (frame[j] == pages[i]) {
                counter++;
                time[j] = counter;
                flag1 = flag2 = 1;
                break;
            }
        }

        if (!flag1) {
            for (int j = 0; j < frameCount; j++) {
                if (frame[j] == -1) {
                    counter++;
                    frame[j] = pages[i];
                    time[j] = counter;
                    flag2 = 1;
                    pageFaults++;
                    break;
                }
            }
        }

        if (!flag2) {
            int pos = findLRU(time, frameCount);
            frame[pos] = pages[i];
            time[pos] = counter;
            pageFaults++;
        }

        printf("Page %d: ", pages[i]);
        for (int j = 0; j < frameCount; j++) {
            if (frame[j] != -1)
                printf("%d ", frame[j]);
            else
                printf("- ");
        }
        printf("\n");
    }

    printf("\nTotal Page Faults = %d\n", pageFaults);
}

int main() {
    int pageCount, frameCount;

    printf("Enter the number of pages: ");
    scanf("%d", &pageCount);

    int pages[pageCount];
    printf("Enter the page reference string: ");
    for (int i = 0; i < pageCount; i++) {
        scanf("%d", &pages[i]);
    }

    printf("Enter the number of frames: ");
    scanf("%d", &frameCount);

    lruPageReplacement(pages, pageCount, frameCount);

    return 0;
}

```

Output:

```

Enter the number of pages: 12
Enter the page reference string: 1 3 0 3 5 6 3 0 1 3 0 2
Enter the number of frames: 3
Page 1: 1 - -
Page 3: 1 3 -
Page 0: 1 3 0
Page 3: 1 3 0
Page 5: 5 3 0
Page 6: 5 3 6
Page 3: 5 3 6
Page 0: 0 3 6
Page 1: 0 3 1
Page 3: 0 3 1
Page 0: 0 3 1
Page 2: 6 3 2
Total Page Faults = 9

```

Optimal

Aim:

To write a c program to implement Optimal page replacement algorithm.

ALGORITHM:

1. Start the process
2. Declare the size
3. Get the number of pages to be inserted
4. Get the value
5. Declare counter and stack
6. Select the least frequently used page by counter value
7. Stack them according the selection.
8. Display the values
9. Stop the process

PROGRAM:

```
#include <stdio.h>
```

```
int isHit(int page, int frame[], int frameCount)
{
    for (int i = 0; i < frameCount; i++) {
        if (frame[i] == page)
            return 1;
    }
    return 0;
}
```

```
int predict(int pages[], int frame[], int
pageCount, int index, int frameCount) {
    int res = -1, farthest = index;
    for (int i = 0; i < frameCount; i++) {
        int j;
        for (j = index; j < pageCount; j++) {
            if (frame[i] == pages[j]) {
                if (j > farthest) {
                    farthest = j;
                    res = i;
                }
            }
        }
        break;
    }
}
```

```

    }

    // If page not found in future, replace
it immediately
    if (j == pageCount)
        return i;
}

// If all pages are found in future
return (res == -1) ? 0 : res;
}

void optimalPageReplacement(int pages[], int
pageCount, int frameCount) {
    int frame[frameCount];
    int pageFaults = 0;
    int filled = 0;

    for (int i = 0; i < frameCount; i++)
        frame[i] = -1;

    for (int i = 0; i < pageCount; i++) {
        printf("Page %d: ", pages[i]);

        // Check if page is already in frame
        if (isHit(pages[i], frame, frameCount)) {
            printf("Hit\n");
        } else {
            if (filled < frameCount) {
                frame[filled++] = pages[i];
            } else {
                int pos = predict(pages, frame,
pageCount, i + 1, frameCount);
                frame[pos] = pages[i];
            }
            pageFaults++;

            // Print frame content
            for (int j = 0; j < frameCount; j++)
{
                if (frame[j] != -1)
                    printf("%d ", frame[j]);

```

```

        else
            printf("- ");
    }
    printf("(Page Fault)\n");
}

printf("\nTotal Page Faults = %d\n",
pageFaults);
}

int main() {
    int pageCount, frameCount;
    printf("Enter the number of pages: ");
    scanf("%d", &pageCount);

    int pages[pageCount];
    printf("Enter the page reference string: ");
    for (int i = 0; i < pageCount; i++) {
        scanf("%d", &pages[i]);
    }
    printf("Enter the number of frames: ");
    scanf("%d", &frameCount);

    optimalPageReplacement(pages, pageCount,
frameCount);
return 0;
}

```

File Organization Technique- Single and Two level directory

AIM:

- To implement File Organization Structures in C are
- a. Single Level Directory
 - b. Two-Level Directory
 - c. Hierarchical Directory Structure
 - d. Directed Acyclic Graph Structure

a. Single Level Directory

ALGORITHM

1. Start
2. Declare the number, names and size of the directories and file names.
3. Get the values for the declared variables.
4. Display the files that are available in the directories.
5. Stop.

PROGRAM:

```
#include <stdio.h>
#include <string.h>

struct Directory {
    char name[20];
    int fileCount;
    char files[10][20];
};

int main() {
    struct Directory dir;
    int i;

    printf("Enter the name of the directory: ");
    scanf("%s", dir.name);

    printf("Enter number of files: ");
    scanf("%d", &dir.fileCount);

    for (i = 0; i < dir.fileCount; i++) {
        printf("Enter name of file %d: ", i + 1);
        scanf("%s", dir.files[i]);
    }

    printf("\nDirectory Name: %s\n", dir.name);
    printf("Files:\n");
    for (i = 0; i < dir.fileCount; i++) {
        printf("%s\n", dir.files[i]);
    }
    return 0;}
```

ALGORITHM:

1. Start
2. Declare the number, names and size of the directories and subdirectories and file names.
3. Get the values for the declared variables.
4. Display the files that are available in the directories and subdirectories.
5. Stop.

PROGRAM:

```
#include <stdio.h>
#include <string.h>

struct SubDirectory {
    char name[20];
    int fileCount;
    char files[10][20];};

struct Directory {
    char userName[20];
    int subDirCount;
    struct SubDirectory subDirs[10];
};

int main() {
    struct Directory dir[10];
    int userCount, i, j, k;

    printf("Enter the number of users (main
directories): ");
    scanf("%d", &userCount);

    for (i = 0; i < userCount; i++) {
        printf("\nEnter name for user %d: ", i +
1);
        scanf("%s", dir[i].userName);

        printf("Enter number of subdirectories for
%s: ", dir[i].userName);
        scanf("%d", &dir[i].subDirCount);
```

```

        for (j = 0; j < dir[i].subDirCount; j++) {
            printf("    Enter name of subdirectory
%d: ", j + 1);
            scanf("%s", dir[i].subDirs[j].name);

            printf("    Enter number of files in
subdirectory %s: ", dir[i].subDirs[j].name);
            scanf("%d",
&dir[i].subDirs[j].fileCount);

            for (k = 0; k <
dir[i].subDirs[j].fileCount; k++) {
                printf("        Enter file %d name: ",
k + 1);
                scanf("%s",
dir[i].subDirs[j].files[k]);
            }
        }

        // Display
        printf("\n===== Directory Structure =====\n");
        for (i = 0; i < userCount; i++) {
            printf("\nUser: %s\n", dir[i].userName);
            for (j = 0; j < dir[i].subDirCount; j++) {
                printf("    Subdirectory: %s\n",
dir[i].subDirs[j].name);
                for (k = 0; k <
dir[i].subDirs[j].fileCount; k++) {
                    printf("        File: %s\n",
dir[i].subDirs[j].files[k]);
                }
            }
        }

        return 0;
}

```